Contents

XO Arena — Performance Plan v2 **1**
Targets (binding budgets) . . . 1
Roadmap at a glance (2026-05-05) . . . 2
Current baseline (2026-05-02 staging, cold-anon) . . . 4
Prod sanity-check baseline (2026-05-04) . . . 4
Pre-flight — before running Phase 0 . . . 6
Post-flight — clean up after the run . . . 7
Phase 0 — Re-baseline . . . 8
Section A — Real performance . . . 9
Section B — Perceived performance . . . 12
Section C — Newly heavy surfaces (deserve their own pass) . . . 13
Section C2 — Compute, cache, and architecture (added from brainstorm) . . . 14
Section D — Instrumentation as a first-class deliverable . . . 15
Section E — Aggressive bets (re-review 2026-05-04) . . . 18
Section F — Pre-launch review gaps (2026-05-04) . . . 21
Appendix Z — Completed measurement work (archive) . . . 31
Out of scope (and why) . . . 35
Sequencing *(updated 2026-05-02 from snapshot data)* . . . 36

XO Arena — Performance Plan v2

v1 lives at `doc/archive/Performance_Plan.md` and is closed out (17 phases, Ready ~330ms floor on the small surface set that existed at the time).

This v2 is a fresh pass against the *current* platform: Fly.io hosting, four services (backend / landing / tournament / packages), SSE+POST realtime, Tables-as-primitive, Multi-Skill Bots (Phase 3.8), the Intelligent Guide journey, and a mobile UI. v1's numbers and assumptions are stale; this plan re-baselines first and prescribes fixes second.

Goal: bear down on real *and* perceived performance across **every** user- facing surface — desktop and mobile, cold and warm, anonymous and signed-in, free play and tournament play.

Targets (binding budgets)

Every page on Fly prod, p75 over a 7-day RUM window:

Metric	Desktop	Mobile (4G mid-range)
FCP	≤ 100ms	≤ 250ms
Ready (spinner gone)	≤ 200ms	≤ 500ms
LCP	≤ 200ms	≤ 500ms
INP (interaction p75)	≤ 100ms	≤ 200ms
TTFB (HTML)	≤ 60ms	≤ 150ms

Backend p95 budgets (per endpoint, single-region):

Endpoint family	p95
`GET /bots`, `/leaderboard`, `/puzzles`	≤ 30ms
`GET /tournaments`, `/tournaments/:id`	≤ 100ms
`GET /users/*` (stats, history)	≤ 80ms
`POST /rt/tables` (create)	≤ 120ms
`POST /rt/tables/:slug/move`	≤ 60ms
SSE channel join → first event	≤ 80ms

Bundle budgets per route:

- Initial parse on `/`: ≤ 120 KB gz JS, ≤ 20 KB CSS
- Largest other route: ≤ +60 KB gz over baseline
- Hero image: ≤ 200 KB on mobile (~~today: 888 KB~~ → **shipped 2026-05-05 at 50 KB mobile / 174 KB desktop** via responsive WebP, see Phase 3)

If a phase's measured win doesn't move at least one budgeted metric, mark it *landed but ineffective* and revisit the assumption.

Roadmap at a glance (2026-05-05)

The single table that says what's shipped, what's queued, and what's deferred — with measured impact, cost, and risk for each major item. Sorted by **(real ms saved × user reach)**. See per-phase sections below for the full design / step list, and Appendix Z for completed- work narratives.

Shipped (2026-05-05)

Item	Real impact (measured)	Cost	Risk	Where it landed
Phase 5.1 — Cold-broker XREAD fix	publishToPickup p95 **740 → 126 ms** at c=5 (−83%); cold first-move p50 **382 → 39 ms** (−90%)	1h	none	v1.4.0-alpha-4.3
F9.1 — pg.Pool max 10 → 30	apply p95 unchanged at this load — no measurable gain yet (still 45-46ms at c=25); pool was not the actual bottleneck	1h	none	v1.4.0-alpha-4.3
Phase 1d — Instant-ready hero	mobile FCP **1416 → 432 ms** (−69%), mobile LCP **1792 → 432 ms** (−76%), desktop FCP **804 → 424 ms** (−47%)	1d	low	v1.4.0-alpha-4.2
Phase 3 — Responsive WebP hero	mobile img bytes **888 → 50 KB** (−94%); mobile TBT 62 → 50 ms	1d	none	v1.4.0-alpha-4.1
F4 — SSE round-trip decomposition (Server-Timing + `_t` breadcrumbs)	enabled the load-test analysis that found the cold-broker bug	0.5d	none	v1.4.0-alpha-3.0
`perf-sse-load.js` — concurrent harness	closes F8 gap #3 (live SSE under load); proved publishToPickup is wakeup-bound, not load-bound	0.5d	none	v1.4.0-alpha-4.3

Active queue — Tier 0 (next sprints)

Cohort-aware sequencing (added 2026-05-05 after F11): the warm-cache baseline showed returning-user mobile Ready already ~370 ms vs cold-anon ~2050 ms. **Phase 1 is a first-time-visitor win**; the returning-user equivalent is **SW + SWR caching**. Both belong in Tier 0 with sequencing decided by which user pain we hit first.

Rank	Item	Real impact (measured / estimated)	Cost	Risk	Cohort	Status
1	**Phase 1** — bundle splitting + per-route lazy load	mobile FCP cold ~1400 → ~700 ms; warm parse ~370 → ~250 ms	3-5d	medium	first-time	queued
2	**Phase 20** — Service Worker app shell	warm Ready 370 ms → ~50-100 ms (paint-only floor)	2-3d	medium	returning	**promoted from Tier 3**
3	**SWR data caching + hover prefetch** (new — see §F11.4)	data pages: spinner-while-fetch → instant stale + bg revalidate	1-2d	low	returning	filed
4	**Phase 17** — per-PR perf gates	locks in Phase 1 win; meta — direct ms = 0	0.5d	none	meta	sequenced after Phase 1

Rank	Item	Real impact (measured / estimated)	Cost	Risk	Cohort	Status
5	**Phase 1c** — cold-authed orchestration (sync dedupe)	cold-authed Ready ~−30%	1d	low	first-time	queued

(Phase 5 remaining steps demoted from Tier 0 — F9 probe sweep 2026-05-05 showed `apply.post` ≤2ms p95 at c=50, so in-process fanout / broker shortcut is not where wall-clock lives. See §F9 and Appendix Z.1.9.)

(F9.2 — VM CPU bump — **deferred 2026-05-05 on cost grounds.** The F9 probes proved it's the right perf lever, but performance-2x VMs add ~$87/mo across staging + prod. Revisit when traffic justifies it (sustained c≥25 in production telemetry) or when Fly's pricing changes. Until then, c≤25 staging perf is "good enough" — apply p95 26ms, movePostAck p50 ~220ms.)

Active queue — Tier 1 (after Tier 0 lands)

Item	Real impact (estimated)	Cost	Risk
Phase 13 — TournamentDetail refactor	heaviest single surface, 1900-line render path	2-3d	medium
Phase 7 — mobile-specific (font preload, touch responsiveness)	mobile FCP/INP polish	1-2d	low
Phase 8 — skeletons everywhere	perceived-only, but high yield on engaged routes	1d	none
Better Auth allow-list trim	cold-authed Ready −80-150 ms	0.5d	low
Phase 5 — remaining steady-state SSE (Nagle, coalesce, Redis hop)	publishToPickup p50 17-21ms — modest, only if F9.2 underwhelms	1-2d	low

Deferred / out-of-scope until evidence changes

Item	Why deferred	Re-promote when
Phase 2 — DB indexes	F9 audit found no missing indexes; query time <20 ms p95	Apply p95 stays >50 ms after F9.2
Phase 6 — backend cold start	Fly auto-stop disabled in iad; not the bottleneck	If we re-enable auto-stop
Phase 14 — Gym worker thread	Non-realtime; lower user-reach than Tier 0 items	After Tier 0 ships
F9.2 — VM CPU bump (1 shared → 2 dedicated)	Probe-validated as the right lever, but ~+$87/mo across staging + prod for performance-2x. Current c≤25 staging perf "good enough" (apply p95 26 ms, movePostAck p50 ~220 ms).	Sustained c≥25 in prod telemetry, or pricing change
Section E aggressive bets (Bun, WebTransport, Hono edge, RSC)	Tier 4 architecture; revisit only if Tier 0+1+2 leaves a gap	Post-V1

Headline answer (was open, settled 2026-05-05)

The F9 probes (granular `apply` Server-Timing + live pool stats) ran on staging 2026-05-05 and gave a definitive read:

- **`pool.waiting = 0` at every concurrency level (1 → 50).** Pool max=30, peak total observed = 9. Pool was *never* the bottleneck; F9.1 retroactively confirmed as a no-op for perf (kept as free hedge for c≥100).
- **`apply.find` ≈ `apply.update`** at every level — both grow proportionally. JSONB write contention was a weaker hypothesis than expected; read latency grows the same way.
- **`apply.post` ≤ 2 ms p95 at every level** — broker dispatch + io.emit + appendToStream is *not* contention. Phase 5 in-process fanout step can be **deferred** with high confidence.
- **`movePostAck` p50 grows 163 → 302 ms** at c=1 → c=50, but `apply` p50 only 8 → 15 ms. Apply is just 5% of the ack at c=50. The other ~290 ms growth lives **outside `applyMove`** — Express middleware, auth, lookupTableForCaller, JSON parse, network. That is the **1-shared-vCPU saturation profile.**

Conclusion: F9.2 (VM bump 1 shared → 2 dedicated) is the **right next perf lever** with hard data behind it (predicted: `movePostAck` p50 at c=50 drops 302 → ~150-180 ms). **Deferred 2026-05-05 on cost grounds** (~+$87/mo across staging + prod for

performance-2x VMs). Current c≤25 staging perf is "good enough" (apply p95 26 ms, movePostAck p50 ~220 ms). Revisit when traffic sustains c≥25 in production telemetry, or when Fly's pricing shifts. The instrumentation stays — when we *do* re-test, the probes already wired in v1.4.0-alpha-4.4 will produce a clean before/after.

See §F9 for the full audit and Appendix Z.1.9 for the staging probe sweep narrative.

Current baseline (2026-05-02 staging, cold-anon)

Full numbers + caveats: `doc/Performance_Snapshot_2026-05-02.md`. The single-pass median across 13 routes:

Metric	Target (desktop)	Measured	Over by
FCP	≤ 100ms	~525ms	5.3×
LCP	≤ 200ms	~640ms	3.2×
Ready	≤ 200ms	~760ms	3.8×

Metric	Target (mobile)	Measured	Over by
FCP	≤ 250ms	~1360ms	5.4×
LCP	≤ 500ms	~1450ms	2.9×
Ready	≤ 500ms	~1580ms	3.2×

Every route is over budget on both devices. Three load-bearing facts from the snapshot drive the priorities below:

1. **One bundle, every page.** First-paint JS is ~493 KB gz on every route (`vendor-react` 82 KB + `main.supported` 411 KB). The `main.supported` chunk is **1,529 KB raw / 411 KB gzip** and contains the entire app — no per-route lazy splitting at the page level today. FCP / LCP / Ready collapse to within ~100ms of each other on every route, the fingerprint of a single bundle blocking the first paint.
2. **PlayVsBot is genuinely 600–700ms slower than the next-worst route** on both devices (10-run confirmation). Not noise. The `/play?action= vs-community-bot` path runs sequential: `getCommunityBot()` → `/api/v1/rt/tables` POST → redirect, all before the spinner clears.
3. **Image bytes were not counted.** `colosseum-bg.jpg` (888 KB) didn't appear in the static byte total — likely loads async after Ready resolves. Need to confirm before promoting Phase 3.

The data does not support chasing DB indexes (Phase 2), SSE latency (Phase 5), or cross-service hops (Phase 4) yet — the bundle + parse floor is currently dominating everything else by a ~10× margin. Those phases stay in the plan but move to Tier 2.

Prod sanity-check baseline (2026-05-04)

To confirm staging is a usable proxy for prod, ran `perf-v2.js` against `xo-landing-prod.fly.dev` (`--target=prod --warmup`, 5 runs × 13 routes × desktop+mobile, cold-anon), then immediately re-ran against `xo-landing-staging.fly.dev` to remove staleness from the comparison.

Saved at:

- `perf/baselines/perf-prod-2026-05-04T10-49-09-639Z.json`
- `perf/baselines/perf-staging-2026-05-04T10-55-42-186Z.json`

Headline: prod ≈ staging within run-to-run noise. Staging is a usable proxy.

Device	Mean Δ% (prod − staging, ready p50)	Notes
Desktop	**+8.6%** (n=13)	Two routes prod-faster (Stats −8%); rest +2 to +19%
Mobile	**−0.1%** (n=13)	Spar prod-faster by 15%; everything else within ±5%

Per-route ready-p50 (ms), prod vs same-day staging:

Route	Desktop prod	Desktop stg	Δ%	Mobile prod	Mobile stg	Δ%
Home	985	878	+12%	2077	2036	+2%
Play	969	849	+14%	2051	2046	+0%
PlayVsBot	1490	1421	+5%	2061	2055	+0%
Leaderboard	968	811	+19%	2076	2053	+1%
Puzzles	850	796	+7%	2127	2051	+4%
Tournaments	906	767	+18%	2069	2059	+0%
Tables	977	883	+11%	2085	2038	+2%
Spar	860	837	+3%	1714	2012	−15%
Stats	843	918	−8%	2046	2053	−0%
Profile	913	811	+13%	2110	2052	+3%
ProfileBots	901	830	+9%	2055	2037	+1%
Gym	860	846	+2%	2059	2077	−1%
Settings	893	826	+8%	2081	2063	+1%

> **Earlier draft of this section reported +23%** because the staging comparison was a 2-day-old baseline (2026-05-02) — staging machines had been warm during that earlier sweep. A same-clock comparison shows mobile is essentially identical and desktop is within first-route cold-start variance. Lesson for the playbook: **always run both environments back-to-back** when comparing.

What this means for the plan

- **Staging is a valid proxy for prod.** Optimizations that move staging numbers will move prod numbers by approximately the same amount. Phase 1/1b acceptance can be measured on staging alone going forward; re-baseline prod once per Tier (not per phase) for confirmation.
- **The bundle + parse floor still dominates on prod.** Same JS weight (490 KB), same FCP/LCP/Ready collapse pattern. Phase 1 (per-route splitting) is still the right next move.
- **PlayVsBot tax is environment-independent.** Both prod and staging show PlayVsBot ~600ms above the rest at desktop p50. Phase 1b's sequential-init flattening applies equally to both.
- **One outlier still worth investigating:** Tournaments desktop p95 hit 6448ms on staging (single run) and 3543ms on prod's earlier run — a cold backend machine handling the first `/api/tournaments` request. `--warmup` only HEADs landing. *Fix:* extend `--warmup` to also hit `/api/version` on backend and `/api/tournaments` on tournament before measuring. File against Phase 6 (cold-start).

Follow-ups still open

- Run a one-off `Home × 5` against `aiarena.callidity.com` to confirm custom-domain TLS does not add measurable latency.
- Re-baseline prod after the next Tier-0 phase lands to confirm the staging-as-proxy assumption holds as the bundle gets smaller.

Pre-flight — before running Phase 0

Cold benchmark numbers are only meaningful if the environment is real and the data is representative. Run this checklist before every measurement pass.

Environment up

All three Fly apps + the staging DB must be reachable:

```
fly status -a xo-backend-staging \
  && fly status -a xo-landing-staging \
  && fly status -a xo-tournament-staging
curl -sf https://xo-backend-staging.fly.dev/api/version && echo OK
curl -sf https://xo-tournament-staging.fly.dev/api/health && echo OK
curl -sfI https://xo-landing-staging.fly.dev/ | head -1
```

Warm the machines (don't measure cold-start as cold-page)

Fly auto-suspends idle staging machines. The first request after suspend includes a multi-second machine boot — that's a *cold-start* number, not a cold-page number, and it'll skew every percentile. Two options:

- **Recommended for a benchmark window:** disable auto-suspend on each app for the duration of the run, then re-enable.

  ```
  for app in xo-backend-staging xo-landing-staging xo-tournament-staging; do
    fly scale count 1 --max-per-region=1 -a "$app"  # ensure 1 machine running
    # disable auto_stop_machines in fly.toml or use the API; revert after
  done
  ```

- **Or:** discard the first run of each route in the script (the warmup pass is built into `perf/perf.js` Phase 0.1 work).

Test data shape

Heavy pages need representative rows. Staging may be sparse. Before the run, seed (or confirm) at least:

- One in-progress 32-participant cup (for `TournamentDetailPage` worst-case).
- One user with 5+ bots, at least 2 of which have a trained ML skill (for `/profile?section=bots` and `/gym` heavy paths).
- ≥ 50 leaderboard entries with a mix of bots + humans (for `/leaderboard` disambiguation render cost).
- ≥ 1 active table (`/tables` list) and 1 spectator-joinable table.

If the data isn't there, document it in the snapshot — *don't pretend the numbers reflect prod-shape*.

Auth

`cold-signed-in` and `warm-signed-in` runs need a working test account on staging:

```
# Required env vars (same ones e2e/qa.mjs uses):
echo "$TEST_USER_EMAIL"   # must resolve
echo "$TEST_USER_PASSWORD"

# Smoke the login flow once before the benchmark:
cd e2e && BASE_URL=https://xo-landing-staging.fly.dev npx playwright test smoke --project=chromium
```

Quiesce other traffic

If anyone else is QAing on staging during the run, the numbers are noise. Coordinate a window or run after-hours.

Snapshot the build

Capture the staging git SHA so the snapshot is reproducible:

```
curl -s https://xo-backend-staging.fly.dev/api/version  # logs commit + version
```

Record it at the top of `Performance_Snapshot_<date>.md`.

Post-flight — clean up after the run

Every change made for the benchmark must be reverted, or the next person to hit staging gets a polluted environment (cost from pinned machines, polluted metrics from seed data, locked test users, etc.). Run this checklist *every* time, immediately after the snapshot is captured.

Restore Fly machine policy

If you disabled auto-suspend for the warm pass, put it back:

```
for app in xo-backend-staging xo-landing-staging xo-tournament-staging; do
  # Either revert fly.toml + redeploy, or use the API to flip
  # auto_stop_machines back to "stop". Confirm via:
  fly status -a "$app"   # machines should return to suspended on idle
done
```

If you scaled machine count up, scale back down.

Remove seeded benchmark data

Anything created *for the benchmark* must be deleted. The `um` CLI is the canonical path so cascades fire correctly (Tournament-Participant rows get cleaned up before User delete, etc.):

```
# Test cup created for TournamentDetailPage worst-case
docker compose exec -T backend node --experimental-transform-types --no-warnings \
  src/cli/um.js tournament:delete <cup-id>

# Bots minted for the heavy /profile + /gym runs
docker compose exec -T backend node --experimental-transform-types --no-warnings \
  src/cli/um.js bot:delete <botId>   # repeat per seeded bot

# Throwaway users (anything tagged isTestUser:true that isn't the persistent
# QA account) — confirm before bulk delete
```

If a Phase 0 helper script seeded the data, it should ship with a paired `--cleanup` flag so the teardown is one command, not a manual list.

Reset RUM ingest

Phase 0.2 wires `web-vitals` → `POST /api/v1/rum`. The benchmark's synthetic runs will dump hundreds of fake samples into `RumSample` and skew the next day's percentiles. Either:

- Tag every benchmark sample with `source: 'synthetic-baseline'` and exclude it from RUM dashboards by default; or
- Delete the rows the run produced:

  ```
  DELETE FROM "RumSample"
  WHERE "createdAt" >= '<run-start-ts>'
    AND "createdAt" <= '<run-end-ts>';
  ```

Same logic for backend RED metrics if they're persisted (Prometheus scrapes are ephemeral, so usually nothing to do there).

Restore CI / config drift

If you flipped any flag or env var on staging for the run (e.g. enabled extra logging, raised a cache TTL to test something, turned off a feature gate), revert before walking away. List them in the snapshot doc so nothing's forgotten.

Sanity check

One last pass to confirm staging is back to its normal shape:

```
fly status -a xo-backend-staging              # auto_stop policy normal
curl -s https://xo-backend-staging.fly.dev/api/version | grep -i version
docker compose exec -T backend node --experimental-transform-types --no-warnings \
  src/cli/um.js list --filter=isTestUser     # only persistent QA rows remain
```

Smoke the e2e suite once to confirm nothing regressed during cleanup:

```
cd e2e && BASE_URL=https://xo-landing-staging.fly.dev npx playwright test smoke --project=chromium
```

If anything fails, the snapshot is suspect — re-run after fixing.

Phase 0 — Re-baseline

v1's `perf/perf.js` ran 6 pages on Railway with 2 services. We now have ~20 routable surfaces on Fly with 4 services and a totally different realtime stack.

0.1 Synthetic baseline (Playwright) *(done — 2026-05-02)*

Shipped in `perf/perf-v2.js`. First baseline captured against staging on 2026-05-02 — 130 measurements (13 routes × 2 device profiles × 5 runs) written to `perf/baselines/perf-staging-2026-05-02T*.json`. Analysis in `doc/Performance_Snapshot_2026-05-02.md`.

- ☒ Extend `perf/perf.js` to cover the full route inventory.
- ☒ Two device profiles: desktop (1280×800, broadband) + mobile (Moto G4 / 4G via CDP throttling).
- ☒ Run against `xo-*-staging.fly.dev`. 5 runs each, p50 + p95.
- ☒ Capture: TTFB, FCP, LCP, Ready, JS bytes, image bytes, requests.
- ☒ Persist results into `perf/baselines/<env>-<ts>.json`.
- ☐ **Open follow-ups** — these are real gaps, *not* blockers for Phase 1; tackle when their inputs become available:
  - ☐ Wire `cold-signed-in` and `warm-signed-in` contexts (needs `TEST_USER_EMAIL` + `TEST_USER_PASSWORD` in env).
  - ☐ Run against `xo-*-prod.fly.dev` once Phase 3.8 is promoted.
  - ☐ Re-run on staging once it bumps past `1.3.0-alpha-8.0` so Sprint 3.8.B + 3.8.C are reflected.
  - ☐ Add INP measurement (one scripted interaction per page — open modal / click row / submit form).
  - ☐ Disable Fly auto-suspend on staging for the run, seed the four representative-data fixtures from the Pre-flight section.

0.2 RUM (real-user monitoring) *(not started — Tier 1)*

- ☐ Wire `web-vitals` into the landing app — POST `{ metric, value, route, deviceClass }` to a new `POST /api/v1/rum` endpoint with sampling (10% in prod, 100% in staging).
- ☐ Server stores into `RumSample` table (route, metric, value, percentile bucket, timestamp, country/region). Add Prisma model + migration.
- ☐ Admin dashboard panel: per-route p50/p75/p95 over 24h / 7d. (Reuse the existing admin metrics scaffold.)
- ☐ Define alert: any route's p75 Ready > 1.5× target for 30min → Slack.

0.3 Backend RED metrics (Rate / Errors / Duration) *(not started — Tier 2)*

- ☐ Add request-duration histogram middleware in `backend/src/index.js` and `tournament/src/index.js`. Bucket per route + status.
- ☐ Expose `/metrics` (Prometheus text format) — Fly already supports scraping.
- ☐ Backfill the Observability dashboard (`doc/Observability_Plan.md`) with p95 panels per endpoint family.
- ☐ Confirm SSE channel join + dispatch latency are instrumented (the realtime postmortem mentions per-channel timing — verify it lights up Grafana).

0.4 Targeted gap measurements (2026-05-04 / 05) *(done — see Appendix Z.1)*

The 0.1 baseline left four open questions. New scripts plug each one: `perf/perf-inp.js`, `perf/perf-sse-rtt.js`, `perf/perf-backend-p95.js`, and `perf-v2.js --extended-resources`.

Headline findings (full tables and details in **Appendix Z.1**):

- **INP** under 32 ms p50 across every measured route on both desktop and mobile — *not a problem today*. (Z.1.1)

- **SSE round-trip** ~577 ms p50 prod is the perceived-perf bottleneck on every move; long pole is Fly Upstash pub/sub. Drives **Phase 5** in Tier 0. (Z.1.2)
- **Multi-machine routing bug** found and fixed via Fly-Replay (commit `0771718`) — 18/20 → 0/20 SSE failures on prod. (Z.1.3)
- **Prod re-baseline 2026-05-05** — backend p95 healthy across the board; the two genuinely load-bearing items are the JS bundle and the (then-) 888 KB hero image. (Z.1.4 / Z.1.5)
- **Hero image** identified as the second-biggest single-byte cost after JS — **Phase 3 shipped** with responsive WebP, mobile cuts 888 KB → 50 KB (−94%). (Z.1.6)

Deliverable: a single `Performance_Snapshot_<date>.md` checked in next to this plan, showing where every route currently sits vs the targets above. *Every subsequent phase compares against this baseline.*

Section A — Real performance

Phases that move the actual latency floor (server time, network, parse, render). Ordered by expected impact-to-effort.

Phase 1 — Bundle audit + per-route splitting *(Tier 0 — start here)*

The 2026-05-02 visualizer pass (run `VISUALIZE=1 npx vite build` in `landing/`) showed `main.supported-*.js` at **1,529 KB raw / 411 KB gz** holding the entire app. Only `vendor-react`, `game-xo`, `game-pong`, and the Gym sub-tabs are split out — every page component is in `main`. Per- route splitting is the highest-leverage fix in the entire plan; until this lands, every other phase is a rounding error.

Concrete sub-steps (in order):

- ☐ **Per-route `React.lazy()` for every page in `App.jsx`.** PlayPage, GymPage, TournamentDetailPage, TournamentsPage, ProfilePage, BotProfilePage, PublicProfilePage, RankingsPage, StatsPage, SettingsPage, TablesPage, TableDetailPage, SparPage, PuzzlePage, plus all `/admin/*` pages. Wrap each `<Suspense fallback={<Skeleton/>}>` so the skeleton already aligns with Phase 8 work.
- ☐ **Vendor-split `@xo-arena/*`** in `manualChunks` — `xo`, `nav`, `ai`, `sdk`. They're currently bundled into `main.supported`. Promote each to its own chunk.
- ☐ **Audit Better Auth client.** Likely a meaningful chunk (auth-client + better-fetch + zod). If the auth flow is only relevant after a sign-in trigger, lazy-load the client module.
- ☐ **Audit `recharts`.** Only used in Gym Analytics + Stats. Confirm `vendor-charts` chunk separation; if it's leaking into `main`, fix the import boundary.
- ☐ **Audit ML engines (`@xo-arena/ai`).** Q-learning / DQN / AlphaZero should only load on Gym Train tab — already lazy at the tab level, but verify the engines aren't statically imported elsewhere.
- ☐ **Audit images / SVG / icons.** Anything imported as a JS module should be inspected; large inline SVGs should move to `<img>` or a sprite sheet (currently inline JSX SVGs ship inside the JS chunk).
- ☐ **Tactical library swaps.** Each of these has a measured-bytes payoff; do them as separate PRs so each can be validated:
  - `recharts` → `uplot` (~20× smaller, only used in Gym Analytics + Stats).
  - Audit `dayjs` boundary; nothing should pull `moment`, `luxon`, etc.
  - Subset `lodash` imports (`lodash-es/get`, not `lodash`); remove `lodash` if any path still uses it.
  - Audit `zod` boundary — only use on validators that actually run client-side.
- ☐ **Tree-shake hygiene.** Mark `sideEffects: false` on every internal `@xo-arena/*` package so unused exports drop. Audit duplicate deps (`npm ls react`, `npm ls zustand`).
- ☐ **Brotli at the edge.** Confirm Fly is serving Brotli, not just gzip — usually 15–20% smaller wire bytes for free.
- ☐ **Aggressive minifier.** Compare `esbuild` vs `swc` vs `terser` passes on `main.supported`. Cumulative ~5–10% gain typical.
- ☐ *(Optional / longshot)* Preact compat alias — saves ~50 KB gz over React 18, but Better Auth + react-router compat needs verification. Land last, behind a feature flag.
- ☐ Generate updated visualizer report; commit at `perf/baselines/bundle-<date>.html` for diffing.
- ☐ Re-run `perf-v2.js` against staging — confirm `main` < 200 KB gz, per-route chunks 30–80 KB gz, and FCP / Ready drop on every page.
- ☐ **CI bundle-size guard.** Fail PR if any chunk grows > 5% over `main` without a `perf-ok` label. (`size-limit` or hand-rolled.)

Expected outcome (data-driven hypothesis): - `main` chunk: 411 KB gz → ≤ 200 KB gz - First-paint JS: 493 KB gz → ~280 KB gz on Home, lower per-route - Mobile FCP: 1360ms → ~700ms (parse cost cut roughly proportional to byte cut on Moto

G4 CPU) - Desktop Ready: 760ms → ~400ms

If the second perf-v2 run does *not* show these moves, Phase 1 is *landed but ineffective* and we go back to the visualizer to find what else is stuck in `main`.

Phase 1d — Instant-ready hero *(shipped 2026-05-05)*

Inlined a static tic-tac-toe board + `<h1>AI Arena</h1>` in `landing/index.html`'s `<div id="root">`. Renders the moment HTML lands; React's `createRoot.render()` clears the placeholder on mount.

Measured impact (staging v1.4.0-alpha-4.2):

- Mobile FCP **1416 → 432 ms (−69%)**
- Mobile LCP **1792 → 432 ms (−76%)**
- Desktop FCP **804 → 424 ms (−47%)**

No bundle parse happens earlier — Ready is unchanged — but the visible "page is alive" moment now lands at the HTML-render boundary. Cost: 1 day; no risks materialised.

Phase 1b — PlayVsBot deep-dive *(Tier 0, immediate after Phase 1)*

The 10-run confirmation showed `/play?action=vs-community-bot` is 600–700ms slower than the next-worst route on both devices, *every time*. The path is sequential and visible to a returning user every time they click "Play vs Bot" cold:

```
mount → getCommunityBot() fetch → /api/v1/rt/tables POST → redirect → render
```

- ☐ Trace each step server-side and client-side; identify the longest synchronous wait.
- ☐ Render an interim board *shell* (skeleton board + "Finding bot…") *before* `getCommunityBot()` returns — the spinner detection then flips when the bot is ready, not 600ms later.
- ☐ Move `getCommunityBot()` cache warming into the route entry — hover-prefetch `/play` from the home CTA, fire the bot fetch then.
- ☐ Verify `/api/v1/rt/tables` POST is the actual path (not socket). Make sure the server returns the table id immediately and SSE backfill picks up state, instead of a single big response.
- ☐ Re-measure with the same `--routes=PlayVsBot --runs=10` invocation. Target: parity with `/play` (≤ 750ms desktop, ≤ 1600ms mobile).

Phase 2 — Database query audit + indexing pass

The schema has grown a lot since v1. Audit slow queries across hot endpoints, especially the new ones:

- ☐ Enable Postgres `pg_stat_statements`; collect 24h of staging traffic.
- ☐ Top 20 by total time + top 20 by mean time — review each.
- ☐ Critical paths to check: tournament list (`/api/tournaments`), tournament detail with rounds + participants + matches eager-loaded, leaderboard, bot list with skills, ML session list, table presence lookups.
- ☐ Add missing indexes (most likely: `TournamentMatch (tournamentId, roundId)`, `TournamentParticipant (tournamentId, status)`, `BotSkill (botId, gameId)` — verify the partial unique still covers reads).
- ☐ Convert any lingering N+1s to `findMany` + grouped indexing (`groupBy`/`include`).
- ☐ **Materialized view for the leaderboard.** The current SQL aggregates win counts on every request. Refresh the view on game-end (already a single hot path). Read becomes O(rows returned), write cost moves out of the request thread.
- ☐ **Read replica for hot reads** (Fly Postgres supports it). Public read endpoints (`/leaderboard`, `/bots`, `/tournaments`) go to the replica; writes + auth stay on primary. Cuts write- lock contention on busy windows.
- ☐ **Stream large responses.** `GET /api/tournaments/:id` currently returns the full eager tree as one payload. With Express `res.write()` chunks (or NDJSON), the client can paint the header → bracket → participant table → match history in stages instead of waiting on the longest one.
- ☐ **Worker queue for off-request work.** ML training kicks, ELO calibration, journey event publishing — every one of these blocks the request that triggered it today. Push to BullMQ-style queue (Redis-backed); the request returns immediately.
- ☐ **Pre-warm critical caches on boot, not first request.** Built-in bot roster, puzzles, system config — load on `app.listen` callback so the first user doesn't pay the miss.
- ☐ **Compress payload responses.** Confirm gzip/Brotli on JSON responses, not just static assets — cuts wire bytes 60–80% for tournament/leaderboard JSON.

Phase 3 — Hero image + asset diet ***(shipped 2026-05-05)***

Hero image diet shipped in v1.4.0-alpha-4.1 — responsive WebP via CSS `@media (min-width: 768px)`, **mobile 50 KB / desktop 174 KB** (was 888 KB JPG, −94% / −80%). Full design, candidates evaluated, and measurement table in **Appendix Z.1.6**.

Remaining open (not blocking — backlog items):

- ☐ Audit other public images (`landing/public/`); drop unused.
- ☐ Add `loading="lazy"` + explicit `width`/`height` on every `<img>`.
- ☐ **Subset fonts** to glyphs actually used (~25 KB vs ~120 KB).
- ☐ **SVG sprite sheet** for icons (currently inline JSX `<svg>` ships inside the JS chunk — moving to an external sprite would let icons stream and cache separately).

Phase 4 — Cross-service network shape

The landing server proxies `/api/(classification|recurring|tournaments)/*` to the tournament service. Each proxied call adds a hop.

- ☐ Measure: cross-service p50/p95 vs same-service.
- ☐ If the gap is meaningful, consider:
  - Direct browser → tournament service via a `tournament.<env>.fly.dev` subdomain + CORS (saves the proxy hop).
  - Or, push read-mostly endpoints (recurring list) into the backend with Redis-backed sharing.
- ☐ Audit `Connection: keep-alive` between landing↔backend, landing↔tournament — don't pay TLS handshake on every request.
- ☐ Confirm **HTTP/3 (QUIC)** is enabled on all Fly fronts. Cuts handshake latency on flaky mobile connections (vs HTTP/2's head-of-line blocking).
- ☐ **Same-origin auth cookie path.** Better Auth sometimes triggers a CORS preflight `OPTIONS` for `/api/auth/get-session`. If the cookie domain is set tight, this is `Access-Control-Max-Age` away from a noticeable savings (or eliminate the preflight entirely).
- ☐ **Audit cookie size.** Better Auth tokens can balloon (esp. with provider claims). Every request carries them — keep < 4 KB.
- ☐ **Combine cold-start API calls into `/api/v1/init`.** A returning signed-in user fires session check + bots + tournaments + journey progress in parallel; one combined endpoint lets the server fan out + return one payload, cutting N waterfalls to 1.

Phase 5 — SSE channel + POST round-trip latency

The realtime layer was rewritten between v1 and v2 (see `doc/Realtime_Migration_Postmortem.md`). It needs its own performance lens:

- ☐ Measure: `EventSource` open → first event latency; channel join → server ack; client POST move → server ack → SSE state event arrival.
- ☐ Look for redundant subscriptions in `useEventStream` (one channel per tab vs one shared singleton — the postmortem mentioned a singleton fix; verify it's still in place).
- ☐ Move idle/heartbeat traffic to a long-poll or coalesce into one channel to keep open connections low (Fly per-instance HTTP limits).
- ☐ Backpressure: under load, what happens when one slow consumer holds an SSE write? Add a watchdog.

Phase 6 — Backend cold start / warm path

Fly machines suspend on idle. First-request latency after suspend can dominate Ready for low-traffic regions.

- ☐ Measure cold-start time per service (backend, tournament, landing).
- ☐ Decide per service: `auto_stop_machines = false` for hot services (backend), keep for cold (admin?). Document the cost trade-off.
- ☐ Pre-warm critical caches (built-in bots, puzzles) on boot, not on first request.
- ☐ Move synchronous init out of the request path (Prisma engine load, journey config load).

Phase 7 — Mobile-specific

Mobile is now first-class but never benchmarked. The 2026-05-02 snapshot showed Mobile FCP at ~1360ms — that's the parse cost on a Moto G4 CPU. After Phase 1 cuts the bundle, the remaining mobile gap will come from critical-path render and CSS.

- ☐ Identify any desktop-only assumption: large tables, hover-only UI, keyboard shortcuts, missed touch targets.
- ☐ Audit DOM size on the heaviest mobile pages (TournamentDetailPage, GymPage with tabs).
- ☐ Lighthouse mobile score ≥ 90 per route as a hard CI gate.
- ☐ **Critical CSS inlined into `<head>`.** Most CSS is render-blocking today (`<link>` tag → server round-trip on bad networks). Extract the above-the-fold subset and inline it; defer the rest. Likely saves 50–150ms on Moto G4 / 4G FCP.
- ☐ **`touch-action: manipulation`** on tappable areas. Eliminates any residual 300ms tap delay; cheap one-liner.
- ☐ **Transform-only animations.** Audit any animation that uses `top`/`left`/`width`/`height` — replace with `transform` so it runs on the compositor thread, not main.
- ☐ **`content-visibility: auto`** on long off-screen sections (the tournament participant table when collapsed, the leaderboard rows below the fold, the Gym session list). Free skip-render for non-visible content; one CSS line per container.
- ☐ **Reduce DOM nodes** on mobile-heavy pages. TournamentDetailPage's 1900 lines render hundreds of nodes — split via Suspense and only mount visible sections.
- ☐ Service Worker for offline-first shell (covered in Phase 21 below).

Section B — Perceived performance

These don't move the latency floor but make the platform *feel* faster. v1 covered skeletons, hover prefetch, optimistic moves, optimistic writes — all good and shipped. v2 extends to the new surfaces.

Phase 8 — Skeletons everywhere new

Every page added since v1 should have a content-shaped skeleton, not a spinner.

- ☐ `TournamentsPage` — list of tournament cards skeletons.
- ☐ `TournamentDetailPage` — header + bracket + participant table skeletons.
- ☐ `BotProfilePage` — bot header + skill cards + ELO chart.
- ☐ `Profile` — accordion frames render with skeletons inside.
- ☐ `Spar` / `Tables` — table card skeletons.
- ☐ Audit: any place where a `<Spinner />` is the first thing rendered after route change is a skeleton candidate.

Phase 9 — Optimistic / streaming everywhere

- ☐ Tournament register: optimistic "Registered □" pill before the server ack (with rollback on NO_SKILL etc.).
- ☐ Bot create: bot card appears immediately with a faded "saving…" badge.
- ☐ Skill add: pill appears in `setBots` optimistic merge (already done in Profile — verify Gym uses the same path).
- ☐ Move events: already optimistic for player; verify bot moves animate smoothly when the SSE event arrives.
- ☐ Streaming responses for large GETs (tournament detail with 100+ matches) — render the bracket as JSON streams in.
- ☐ **Partial-UI loading states.** Replace `Loading…` with progress narration: "Found bot, building table…" / "Bracket ready, fetching participants…". Same wait time, lower perceived latency.
- ☐ **Animation cover during fetch.** A 200ms fade/slide on route change can hide a 200ms fetch entirely — the wait reads as polish.

Phase 10 — Hover-intent prefetch + "warm next" patterns

v1 added hover prefetch on nav links. Extend:

- ☐ On `/tournaments`, hover-prefetch each tournament's detail JSON.
- ☐ On `/profile?section=bots`, hover-prefetch `/gym?bot=:id` chunks for each row.
- ☐ On match end (post-cup), prefetch the next likely route (next match, coaching card).
- ☐ On `/play`, after first move, prefetch the rematch flow's chunks.
- ☐ **Pre-render the next likely route** in a hidden iframe / shadow DOM during idle time on slow paths (post-cup celebrations, match results). Click feels instant.

Phase 11 — Route transition polish

- ☐ Re-evaluate the v1 fade transition under React 18 Suspense — does it conflict with streaming UI?
- ☐ Add `view-transitions` API where supported (Chrome) for snap-zero route transitions on the same-document navigations.

Phase 12 — In-game feel

- ☐ Confirm sound latency is still nailed (the capture-phase pointerdown listener in `soundStore.js` — per CLAUDE.md, must not be removed).
- ☐ Animation budgets: every move, popup, modal — none > 200ms.
- ☐ Reduce-motion respect on every animation.

Section C — Newly heavy surfaces (deserve their own pass)

Phase 13 — Tournament page perf

`TournamentDetailPage` is the heaviest page in the app (1900+ lines, eager loads rounds + participants + matches + their users). It runs before / during / after the cup journey and shows under load on Cup days.

- ☐ Hard target: cold mobile Ready ≤ 600ms even with 32 participants and a full bracket.
- ☐ Split the page: bracket + participant table + match history are independent, render each behind its own Suspense boundary.
- ☐ Server: paginate completed matches; only the current round needs full detail.
- ☐ Memoize bracket layout math; profile re-renders when SSE events fire.
- ☐ Confirm `ParticipantTable` disambiguation (just shipped in 3.8.C) doesn't do per-render allocation.
- ☐ **`useDeferredValue`** on the bracket + participant lists when SSE pushes a new event. Render the new state at lower priority than user input (React 18).
- ☐ **Memoize `BracketMatch`** — currently re-renders on every SSE event for the whole bracket, even matches that didn't change.
- ☐ **`content-visibility: auto`** on rounds the user has scrolled past or off-screen.

Phase 14 — Gym page perf + ML compute

The Gym detail panel re-mounts whole tab trees on bot/skill switch, and ML inference today runs on the main thread (jank during play).

- ☐ Ensure keep-alive (`display: none`) survives the new bot→skill drilldown.
- ☐ ML model fetch should stream — show training history first, then current model weights.
- ☐ **Web Worker for ML inference.** Offload `getMoveForModel` for ML bots to a worker so the main thread never janks during a move.
- ☐ **WASM minimax / AlphaZero.** Could move bot move latency from ~30ms to <5ms. Compile the minimax engine to WASM behind a flag; benchmark vs JS. Bigger payoff once Connect4 lands (state space grows).
- ☐ **OffscreenCanvas for game render** (where applicable). Frees main thread for UI during high-FPS games like Pong.
- ☐ **`requestIdleCallback`** for journey events, telemetry, and RUM flushes — never compete with user input.

Phase 15 — Intelligent Guide journey

The journey adds bus events, popups, scrim spotlights, and re-renders on every step trigger. Measure — it's a likely INP regressor.

- ☐ INP audit on each step trigger; confirm < 100ms.
- ☐ Memoize `JourneyCard` re-renders.
- ☐ Lazy-load step-specific components (TrainGuidedModal, BotCreatedPopup).
- ☐ Confirm guide store mutations don't cascade through the whole tree (split selectors).

Phase 16 — Tables-as-primitive overhead

Tables underlie spar, journey demos, PvB, PvP, and tournament HvB. Verify the common path is hot:

- ☐ `tablePresence` lookups: O(1) per join? Memory footprint at 1000 active tables?
- ☐ `botGameRunner`: parallel runs OK at 50 concurrent demo tables?
- ☐ `createTableTracked` resource counter overhead — single Redis call or multi-hop?

Section C2 — Compute, cache, and architecture (added from brainstorm)

These are bigger swings than the per-phase tactics above. Each one is a "could be a project" rather than a checklist item, but they're listed here so the option is visible when the data points to them.

Phase 19 — Compute relocation

Today, all heavy compute runs on the user's main thread:

- ML training (Q-learning, DQN, AlphaZero) — browser-only.
- ML inference (`getMoveForModel`) — main thread.
- Game logic (minimax, AlphaZero MCTS) — main thread JS.
- Journey events, ELO calibration — request thread on backend.

Targets after this phase:

- ☐ **Web Worker for ML inference.** Offload to a worker; main thread never janks during a move. Pairs with Phase 14.
- ☐ **WASM for minimax / AlphaZero.** Compile the engines to WASM behind a feature flag; benchmark vs the JS engines. Expected 10–100× faster move computation. Critical for Connect4 (tied to `doc/Connect4_Ship_Checklist.md` Spike A on TF.js).
- ☐ **OffscreenCanvas for game render.** When games like Pong land, keep the render loop off the main thread.
- ☐ **Move ML training to a backend worker.** Tracked in `doc/Connect4_Ship_Checklist.md` Spike B — perf is one input; architectural decision is broader (mobile training, headless cluster runs, training on devices that can't keep a tab open).
- ☐ **`requestIdleCallback`** for journey events, telemetry, RUM flushes — never compete with user input on the main thread.

Phase 20 — Cache & app shell

Today: warm visits read from `localStorage` for some stores, but the HTML / JS / CSS still hits the network every time.

- ☐ **Service Worker app shell.** Cache the index HTML + the vendor chunks; serve from cache on second visit; revalidate in background. Repeat-visit Ready ≈ 0ms. Highest-leverage perceived-perf change not yet in v2.
- ☐ **IndexedDB for tournament/leaderboard JSON.** Faster than localStorage at size; survives reload. Stale-while-revalidate directly off IDB.
- ☐ **Edge response cache** on Fly for public reads (`/leaderboard`, `/bots`, `/tournaments`). Backend never sees cached requests.
- ☐ **HTTP `immutable` cache headers** on hashed asset URLs (`max-age=31536000, immutable`). Confirm Fly is doing this.
- ☐ **Persist Zustand stores** so warm boot has full state — guide progress, sound prefs, ui sort orders, etc.

Phase 21 — Architecture experiments (longshots)

Each is a meaningful re-architecture; pursue only if Tier 0 + Tier 1 data shows the gap they'd close. Listed for completeness.

- ☐ **Server-side render the landing page (`/`) only.** It's mostly static. React 19 RSC or a tiny SSR shell could deliver Ready ≈ TTFB. Other routes stay SPA.
- ☐ **Edge functions for `/leaderboard` / `/bots`.** Fly Functions could serve these from cache without touching the Node process, cutting cold-start exposure entirely.
- ☐ **Replace JSON-over-HTTP with gRPC-Web** for hot endpoints — smaller payloads, schema validation, generated clients.
- ☐ **Migrate hot read endpoints off Express to Hono on the edge** — lower per-request overhead than full Express middleware stack.
- ☐ **React Server Components for static-ish pages** — Settings, Puzzles, parts of Profile. Cut their JS to ~0 KB.

Section D — Instrumentation as a first-class deliverable

Performance work without instrumentation is fishing in the dark. v1 measured synthetically once and stopped. v2 makes measurement continuous.

D0 — Pre-Tier-0 instrumentation (landed 2026-05-04, `dev`)

Before committing to Tier 0 work, the toolbox is filled in so each tackled item is *measurable* before and after. Synthetic harnesses that exist today on `dev`:

Aspect	Script	What it answers
Cold-page totals	`perf/perf-v2.js`	TTFB / FCP / LCP / Ready per route × device, with `--extended-resources` for late-loading bytes
Interaction (INP)	`perf/perf-inp.js`	p50/p95 INP per common interaction × route
Backend hot endpoints	`perf/perf-backend-p95.js`	p50/p95/p99 for `/api/version`, `/api/v1/bots`, `/api/v1/leaderboard`, `/api/auth/get-session`, `/api/tournaments`
SSE round-trip (F4)	`perf/perf-sse-rtt.js`	Splits POST move → SSE event into server.lookup, server.apply, redis publish→pickup, broker pickup→write, network legs
Cold-page waterfall (F2)	`perf/perf-waterfall.js`	When the route's primary `/api/*` fires + how long it takes — sizes the prize for `<link rel=preload>` and inline initial payload
Long-task profile	`perf/perf-longtasks.js`	count / sum / max long tasks per cold load (Moto-G4 + 4× CPU throttle for mobile) — sizes Phase 1 / 1b TBT prize
Bundle composition	`perf/perf-bundle.js`	Per-chunk raw / gzip / brotli for every dist/ asset; checked-in baseline for diff after Phase 1

Backend-side instrumentation: - Global `Server-Timing: handler;dur=X` middleware on every backend response (preserves route-level timings like the move POST's `lookup, apply`). CORS exposes `Server-Timing`. The waterfall + any future ad-hoc test can read server vs network breakdown for every `/api/*` call without per-handler instrumentation. - Move POST emits `lookup;dur=X, apply;dur=Y`. - SSE broker injects `_t: { publishToPickupMs, pickupToWriteMs }` into every payload (publishMs comes free from the Redis stream id).

Key bundle finding: `main.supported.js` is **1506 KB raw / 319 KB brotli** — half of total brotli bytes shipped. This is the F5 / Phase 1 target; the rest of the JS budget (16 chunks) is mostly small named splits (TrainTab, ExplainabilityTab, game, etc.).

D1 — RUM (Real-User Monitoring) (landed 2026-05-04, `dev`)

Synthetic baselines see what one Playwright run on a controlled connection sees; RUM sees what real users on real networks actually experience.

- **Client:** `landing/src/lib/rum.js` registers `onFCP / onLCP / onINP / onCLS / onTTFB` from `web-vitals`. Each metric appends to a per-tab queue; on `pagehide` (or first `visibility:hidden`) the queue is drained as a single beacon to `/api/v1/perf/vitals` via `navigator.sendBeacon` with explicit `application/json` content type (the default `text/plain` is silently dropped by Express's body parser — easy footgun, captured here so we don't repeat it). Sampling: `VITE_RUM_SAMPLE_RATE` (default `1.0`); decision is sticky for the tab.
- **Backend:** `POST /api/v1/perf/vitals` (route `backend/src/routes/perfVitals.js`). No auth, no SSE-session required. Validates each entry (allowed name set, finite numeric range 0–1e7, capped 32 vitals per beacon), persists via Prisma `createMany`, always returns 204. 8 vitest cases cover happy path, unknown names, garbage values, oversize beacons, DB failure.

- **Schema:** `PerfVital` table — anonymous (`sessionId` is a per-tab random hex, never a user id), no IP. Indexes on `(name, route, createdAt)` and `(env, name, createdAt)` so the obvious aggregations (p50/p75/p95 per route × device × env over time) stay cheap.
- **Smoke:** Playwright headless run hits `/`, fires `visibilitychange → hidden`, beacon arrives 204, three rows land in DB (TTFB, FCP, CLS — LCP/INP need real interaction to register).

Privacy boundary: never sends `userId`, never stores IP, no fingerprinting. The only durable identifier is the tab-scoped `sessionId`, which has no cross-tab persistence.

Next: ship to staging + prod with the next `/stage` cycle. After ~1 week of real-user data — or immediately, with the synthetic driver in D2 — we have an honest p50/p75/p95 baseline to diff every Tier 0 deploy against. Admin dashboard for it landed 2026-05-04 (see D3).

D2 — Synthetic RUM driver *(planned)*

D1 only fills `perf_vitals` when real humans load pages. Staging has near-zero organic traffic; prod takes hours-to-days to accumulate stable percentiles. So before we can compare a Tier 0 change against a "clean" baseline we need a way to *guarantee* a complete sample set. The synthetic driver is that guarantee — Playwright-scripted sessions that exercise every route on both desktop and mobile profiles and let the same `web-vitals` listeners + beacon do the reporting.

Script: `perf/perf-rum-driver.js` (TBD).

- Launches N parallel `chromium` contexts (default 5; configurable).
- Iterates the standard route inventory (`/`, `/play`, `/tournaments`, `/rankings`, `/gym`, `/profile`, …) per device profile (desktop, mobile w/ 4× CPU throttle via CDP, optionally 4G / 3G network throttle).
- Per route: load → wait for LCP candidate → perform a representative interaction so INP fires (button click that doesn't navigate) → trigger `visibilitychange:hidden` so the beacon flushes → close context. Passive page loads alone don't yield LCP / INP / CLS, so the interaction step is non-optional.
- Tags every session with a recognizable UA suffix (`XO-Synthetic/1.0`) so the aggregation endpoint can split organic vs synthetic via `userAgent ILIKE '%XO-Synthetic/%'`. No schema change, no separate ingest path — same `POST /api/v1/perf/vitals`.
- Backend `GET /admin/health/perf/vitals` gains a `?source=organic|synthetic|all` filter (default `organic`) so the admin dashboard can show either view without polluting the real baseline.

Cadence:

- *On-demand* (always available): developer runs `BASE_URL=https://xo-frontend-staging.fly.dev node perf/perf-rum-driver.js` before and after a change to capture before/after percentiles in one session.
- *Scheduled* (CI): a GH Actions cron drives staging every ~6h so the staging RUM dashboard never shows empty bars. **Prod is not on the cron** — synthetic samples on prod would skew real-user numbers, so prod runs are manual-dispatch only, used when "what does INP look like under prod backend load?" is the actual question.

Caveats:

- Synthetic INP is bounded by Playwright's input-event latency, which is tighter than a real human. Treat synthetic INP as a *stability / regression* signal, not as an absolute number for budget compliance.
- Network conditions skew the picture if not pinned. Default to one desktop + one mobile-throttled profile per run; optionally include 3G / 4G via CDP `Network.emulateNetworkConditions` for the long-tail.
- Synthetic rows accumulate cost in `perf_vitals`. Cleanup is a scheduled prune (D2.1) — see below — not a per-run delete, so comparison runs across days remain possible.

D2.1 — Synthetic-row retention & data hygiene *(planned)*

The synthetic driver produces a continuous trickle of rows; without a prune, `perf_vitals` grows linearly forever (a 6h staging cron writing ~5 routes × 2 device profiles × ~5 vitals = ~50 rows/cycle = ~200 rows/day = ~75k rows/year per cron alone, before any prod synthetic or any organic). Cheap on day one, painful to fix at year three. So the prune ships with D2.

Goals.

1. Keep `perf_vitals` bounded so admin queries stay sub-100 ms even at year-scale.
2. Never delete data mid-experiment — a comparison run that started yesterday must still have its baseline rows tomorrow.
3. Default behaviour is safe: organic rows are untouched, synthetic rows are kept long enough to span a typical Tier 0 PR

cycle.

What gets pruned (and what doesn't).

Source	Default retention	Rationale
Synthetic (`userAgent ILIKE '%XO-Synthetic/%'`)	14 days	Long enough to span a feature branch + review + post-merge diff. Tunable up.
Organic (everything else)	unlimited	Real-user rows are scarce on staging and irreplaceable on prod. Only prune if the table actually outgrows its index — measured, not pre-emptive.
Smoke / dev rows (`env = 'local'`)	7 days	Kept short — they're noise once the smoke run that wrote them has been verified.

Cadence. Nightly. Implemented as a recurring `scheduledJob` (type `perf_vitals_prune`) registered with the existing `scheduledJobs.js` dispatcher: handler runs the deletes, then enqueues the next run 24 h out. If the dispatcher misses a tick (process restart, env unavailable), startup recovery resets stuck RUNNING jobs and the next tick catches up — no lost data, no double-prune.

Implementation sketch.

- New handler module `backend/src/services/perfVitalsRetentionService.js` exporting `pruneSynthetic({ olderThan })`, `pruneSmoke({ olderThan })`, and a `runPrune()` orchestrator. Each delete uses the existing `(env, name, createdAt)` index — an `EXPLAIN` against the staging DB before merge confirms an index scan, not a seq scan.
- Bootstrap from `backend/src/index.js`: register the `perf_vitals_prune` handler, then enqueue the first run if no PENDING / RUNNING row already exists. Idempotent across restarts.
- Test coverage: unit tests for each prune function (boundary on the cutoff, no-op when nothing matches, error path doesn't tombstone the job), plus a handler-level test that asserts the next run is re-enqueued.

Pause / extend for long comparison runs. Two system_config keys (`getSystemConfig` / `setSystemConfig` already wired through the admin panel):

- `perf.retention.synthetic.days` (number, default `14`) — bump to `60` before a multi-week experiment, drop back to `14` afterwards.
- `perf.retention.paused` (bool, default `false`) — hard pause; the handler short-circuits to a no-op while still re-enqueueing itself so resuming is just flipping the flag back. Useful when a multi-day "hold everything" capture is in flight.

Both are read at the top of the handler so a flip takes effect on the next tick (≤ 24 h) without a backend restart.

Observability. Each run logs `{ prunedSynthetic, prunedSmoke, keptOrganic, durationMs, paused }` at info level. The numbers also land in the metrics snapshot so the admin Health dashboard's `/admin/health/sockets` history shows whether the table is stabilizing or still growing.

One-time scrub (only if needed). If any synthetic rows leak into the table *before* D2 ships its UA tag, they'd be indistinguishable from organic. Mitigation:

1. Ship D2 (UA tag + `?source` filter) and D2.1 (prune) **in the same PR** — this is the primary defence; there's no untagged-synthetic period.
2. Belt-and-suspenders: a one-shot admin endpoint `POST /admin/health/perf/vitals/scrub-synthetic-before` accepting a timestamp, runnable manually if step 1 ever fails. Behind `requireAdmin`, idempotent, returns the row count it deleted. Add only if step 1's pre-merge testing reveals a gap; otherwise skip.

Future: dedicated `source` column. The UA-suffix filter is fine at year-one scale (`userAgent` is already indexed via the `(name, route, createdAt)` composite for the hot path; the source filter only matters on infrequent admin queries). If the synthetic volume ever rises to where the `ILIKE` becomes hot, add a `source enum('organic', 'synthetic', 'smoke')` column and a backfill — but not before, since the column adds write-side cost on every beacon and the current shape is already costly enough at hundreds of rows/sec organic.

Sequencing — when does cleanup actually happen? Three distinct moments to keep separate:

1. *Before the next `/promote`* (i.e., right now): **no cleanup needed.** D1 just landed on dev; prod's `perf_vitals` is empty. The `/stage` + `/promote` we're queueing pushes code, not data.
2. *Between `/promote` and the synthetic driver going live*: **also no cleanup needed.** Real users start writing organic rows immediately; that's exactly what we want. The perf-* scripts re-baseline produces JSONs in `perf/baselines/`, never touching `perf_vitals`.
3. *Once D2 + D2.1 land*: cleanup becomes ongoing — the prune does its job nightly, queries default to `?source=organic`, the dashboard stays clean. **No "cleanup phase" between push and data collection** — the cleanliness primitives ship inside D2 itself, not as a separate gate.

D3 — Admin Health dashboard panels (landed 2026-05-04, `dev`)

The on-call surface for everything D0–D2 produces, all under `/admin/health`:

- **Real-User Web Vitals** — `GET /api/v1/admin/health/perf/vitals` — per-(route, metric) p50/p75/p95 with rating-mix bars, 1h/24h/7d toggle, env breakdown. Color-coded against the web-vitals thresholds (LCP ≤ 2.5s good / > 4s poor, INP ≤ 200ms / > 500ms, CLS ≤ 0.1 / > 0.25, …).
- **Perf Baselines (dev-only)** — `GET /api/v1/admin/perf/baselines{,/:filename}` — read-only browser of `perf/baselines/*.json` with kind filter and click-to-view JSON. Disabled on Fly.io (no `PERF_BASELINES_DIR`) because the JSONs only exist on the dev machine that ran the perf scripts. Strict filename regex + resolved-path check guard against traversal.

Tests: 6 + 8 vitest cases on the backend; UI is straightforward presentation.

Phase 17 — Per-PR perf gates

- ☐ CI runs `perf/perf.js` against a preview env on every PR; comments the diff vs `main`.
- ☐ Bundle-size guard fails PR if any chunk grows > 5% without an explicit `perf-ok` label.
- ☐ Backend test suite asserts p95 < budget for the 5 hottest endpoints (run against an in-memory load generator).

Phase 18 — Production perf dashboard

- ☐ Public-internal Grafana page: per-route Ready p75/p95 (RUM), per-endpoint backend p95, SSE channel health, error budgets.
- ☐ Weekly "perf review" cadence — owner reviews against budgets, files issues for any breach > 7 days.

Section E — Aggressive bets (re-review 2026-05-04)

A second pass over the plan exposed several large-leverage ideas that weren't in any phase. Some are nearly-free additions to existing phases; others are real moonshots. They're grouped by what they buy you, not by tier — surface them where they fit, then promote the ones the data backs after Tier 0 lands.

E1 — Free wins (add to Tier 0 / quick-wins)

These cost ≤ a day each, ship behind feature flags if needed, and should be done alongside the bundle work — most don't even require a benchmark to justify.

- ☐ **103 Early Hints from Fly.** Send `<link rel="preload">` for `vendor-react` + `main` + critical CSS *before* the backend has finished computing the HTML response. Saves 50–150ms by overlapping TTFB with preload. Fly's edge supports this; verify and wire it up.
- ☐ **Speculation Rules API** (`<script type="speculationrules">`). Chrome / Edge will prerender same-origin links the user is likely to click — no JS, no library. Add a JSON block with the home, `/play`, `/leaderboard`, and `/profile` URLs. Free instant navigation on Chromium.
- ☐ **HTTP/3 0-RTT for returning visitors.** Cuts the TLS handshake to zero on warm connections. Fly supports HTTP/3; confirm 0-RTT isn't disabled for safety reasons we don't actually need.
- ☐ **Resource hints for auth providers** — `<link rel="preconnect">` / `dns-prefetch` for `accounts.google.com` and `appleid.apple.com` in `<head>` so the OAuth round-trip starts warm. Sign-in feels ~200ms faster on cold visitors who click immediately.

- ☐ **Brotli quality 11 at build** (vs Vite's default 6). Slower build, zero runtime cost, ~5–10% smaller JS payloads. Set `compress: { algorithm: 'brotliCompress', compressionOptions: { level: 11 } }` in the Vite config.
- ☐ **CSS `image-set()`** for the hero — `background-image: image-set("hero.avif" 1x type("image/avif"), "hero.webp" 1x)`. Browser picks best format automatically; supersedes manual `<picture>` wrapping for CSS backgrounds.
- ☐ **CSS `aspect-ratio`** on every `<img>` and lazy section. Locks layout before bytes arrive — kills CLS on the home, leaderboard, and tournament pages without manual `width` / `height`.
- ☐ **BlurHash placeholders** for the hero, bot avatars, and any future user uploads. ~30 bytes per image, decoded synchronously to a 32×32 canvas — instant visual content under the real image.
- ☐ **Variable fonts.** Replace separate Inter regular/semibold/bold static files with one variable font (`Inter.var.woff2`). Roughly half the font weight on the wire.
- ☐ **`font-display: optional`** for non-critical text (e.g., Inter Tight on hero headlines). Never blocks render, even on slow networks.
- ☐ **Modern-only build by default.** Drop the legacy bundle in production (Vite still builds it). 99% of traffic is evergreen Chromium / Safari / Firefox; the legacy chunk is dead weight.
- ☐ **`scheduler.yield()`** in any loop > 5ms (bracket layout, journey derivations, leaderboard sort). Hands control back to the browser between chunks of work — keeps INP under 100ms even on Moto G4.
- ☐ **`isInputPending()`** as the cheap version when `scheduler.yield` isn't shipped: pause heavy work mid-loop if the user is touching the screen.

E2 — React re-render lockdown

The existing phases mention `useDeferredValue` and `React.memo` on `BracketMatch`, but the platform has a deeper render-cost story.

- ☐ **Atomize Zustand stores.** The current game store, journey store, and ui store each have many subscribers. Splitting each into per-domain slices (e.g., `tableStore`, `seatsStore`, `boardStore`) means an SSE state event re-renders only what actually changed. Easy diff pattern; large measurable win on TournamentDetail under live updates.
- ☐ **`useTransition` on every nav click.** Keeps the current page interactive while the next route's chunks load — visible Ready stays low even when the next page is heavy.
- ☐ **Selective hydration where applicable.** React 18 already supports it; needs explicit `<Suspense>` boundaries to opt in. Pair with Phase 1's per-route splitting.
- ☐ **Why Did You Render audit** on TournamentDetailPage, GymPage, RankingsPage. Run in dev for a session, fix every flagged cascade. One-time effort, perpetual savings.
- ☐ **React 19 Actions + `useFormState`** for sign-in, register, bot-create, skill-add. Cleaner optimistic UI and built-in pending state — replaces hand-rolled patterns.
- ☐ **React Forget (compiler memoization)** — track stability; adopt when the React team marks it production-ready. Removes most `useMemo` / `useCallback` boilerplate, often catching cases hand-written memoization missed.

E3 — Compute moonshots (Phase 14 / 19 supercharger)

The existing plan moves ML to a Web Worker. The real wins are bigger.

- ☐ **WebGPU backend for TF.js.** Newer than the WebGL backend, ~2–5× faster on supported hardware. Falls back to WebGL automatically. One config change in TF.js setup.
- ☐ **WASM SIMD for minimax.** Compile the minimax engine with the `simd` flag — 4–8× speedup over scalar WASM, which itself is ~10× faster than the JS version. Bot move latency goes from ~30ms to ~1ms; matters more for Connect4 + AlphaZero.
- ☐ **Quantize ML models to int8.** TensorFlow.js supports int8 quantization at the conversion step. 4× smaller weights on disk + on the wire, 2–4× faster inference on CPU. Trade is ~1–2% accuracy, which is well within ELO noise for our game.
- ☐ **Cache compiled WASM in IndexedDB.** WebAssembly.Module is structured-cloneable. Compile once, store, retrieve on next boot — saves the JIT pass on every reload.
- ☐ **Background-train ML bots while idle.** When the user is reading the home page or watching a demo, run a few training epochs in a Web Worker. Their bot quietly improves between sessions.
- ☐ **Speculative bot move computation.** During the user's turn, precompute the bot's response to the top 3–5 likely user moves. User makes their move → bot's response is already cached → animate immediately.

E4 — Network moonshots

- ☐ **PgBouncer in front of Fly Postgres.** Connection pooler that handles transient connection storms (deploy-time, cold-start flap). Cuts p95 tail latency on DB-bound endpoints. Standard Postgres infra; runs as a sidecar machine.
- ☐ **DataLoader for N+1 batching.** Especially on tournament-detail eager loads — fan out N participant lookups → one batched `SELECT … WHERE id IN (…)`. Battle-tested Node pattern; drop-in.
- ☐ **WebTransport (HTTP/3 datagrams) replacing SSE.** Lower latency than SSE, better connection migration when mobile users switch networks, supports unreliable channels for fire-and-forget. Big lift; do only after RUM data shows SSE is the bottleneck.
- ☐ **WebRTC peer-to-peer for PvP** once tables are seated. Server only relays signaling; gameplay traffic is browser-to-browser. Move latency drops to one round-trip on the same network. Big lift; meaningful only for PvP, not PvB.
- ☐ **Edge KV for read-heavy public reads.** Built-in bots, system config, journey config — load once at the edge, serve from memory; backend never sees these requests. Kills the request class entirely. Fly supports this via a small KV sidecar or Cloudflare Workers KV.
- ☐ **Stale-while-revalidate everywhere safe.** `Cache-Control: max-age=60, stale-while-revalidate=600` on `/leaderboard`, `/bots`, `/tournaments` list. Browser shows cached instantly, fetches fresh in background. One-line back-end change.
- ☐ **Compression dictionaries.** Brotli supports shared dictionaries across requests — train a dictionary on common JSON shapes (tournaments, bots, leaderboard) and ship it. 30–50% smaller on the wire after the first request. Browser support arriving 2026.

E5 — Cache & shell expansion (Phase 20 supercharger)

The existing Phase 20 mentions Service Worker app shell. Push it harder.

- ☐ **Service Worker as the canonical realtime substrate.** SW holds one shared SSE connection for all open tabs of the app — instead of N tabs × 1 SSE = N connections, it's 1 connection multiplexed via `BroadcastChannel`. Fly per-IP connection limits stop biting.
- ☐ **Background Sync for offline moves.** User makes a move on a flaky network → SW queues it → replays when connection returns. Player never sees a failed move.
- ☐ **IndexedDB for SSE event replay.** SW persists every state event to IDB; on tab reopen / reconnect, replay all missed events locally before the live SSE catches up. Reload feels instant.
- ☐ **Periodic Background Sync** for tournament wakeups. SW pings `/api/tournaments/upcoming` every 15 minutes when offline; pushes a notification when one starts. PWA primitive, free.
- ☐ **Push notifications for cup events.** Already partially wired via `pushSubscribe.js` — extend to "your match is starting" and "your bot won" pings.
- ☐ **WebShare API** on results pages so users share their cup wins to native iOS/Android share sheets. Not perf-direct, but cuts the friction loop that drives engagement → repeat sessions where the SW shell pays off.

E6 — Predictive perf

The platform has rich session data; use it to make navigation feel psychic.

- ☐ **Predictive prefetch from user history.** Track per-user navigation transitions (e.g., "users who view profile next view leaderboard 80% of time"). Prefetch the top-1 next route on every page load. Costs one tiny model + a few KB of state.
- ☐ **Pre-render the *expected* next route** in a hidden subtree. On `/play` cup match end, the next click is almost always "Continue" or "Rematch" — both routes can be fully mounted and ready when the click happens.
- ☐ **Speculative API calls based on cursor proximity.** As the cursor approaches a tournament card, fire `/api/tournaments/:id` before the click. Hover-prefetch as a behavior model, not just a CSS pseudo-class.

E7 — Architecture longshots (Tier 4)

Real re-architectures. Each is a project. Listed for completeness so the option is visible.

- ☐ **Bun runtime for backend + tournament services.** Cold-start drops from ~500ms (Node) to ~50ms (Bun) and request throughput goes up ~30%. Bun is Express-compatible; the migration is mostly testing. Collapses Phase 6 (cold-start) entirely. Risk: Prisma + Bun stability, Better Auth compatibility.
- ☐ **Hono + edge runtime for read-only public APIs.** Move `/leaderboard`, `/bots`, `/tournaments` list to Hono on Fly's edge. ~10× faster per request than Express, runs in every region. Already in Phase 21 — promote if Bun isn't ready.

- ☐ **Cloudflare Durable Objects (or Fly Replay) for tablePresence.** Move the in-memory presence cache to a regional state primitive. Frees backend machines to scale horizontally; eliminates a hot shared-memory table.
- ☐ **gRPC-Web for hot endpoints.** Schema-validated, smaller payloads, generated clients. Already in Phase 21; revisit after Bun.
- ☐ **Replace Better Auth with self-rolled JWT + refresh.** If the cookie-size + preflight overhead in Phase 4 measurements turns out to be load-bearing, the entire auth library may be the cheapest thing to swap. High-risk; only with measurement.

Section F — Pre-launch review gaps (2026-05-04)

A second pass before starting Tier 0 surfaced gaps the existing phases don't cover, plus DB / REST patterns the user explicitly asked about that the plan was silent on.

F1 — DB join shape + preloading (Phase 2 supplement)

Phase 2 calls for an index pass + `pg_stat_statements`, but doesn't audit the *join shape* of the heaviest endpoints. Hot offenders:

- **`GET /api/tournaments/:id`** — three-level `include`:

  ```
  include: {
    participants: { include: { user: { select: { …8 cols } } } },
    rounds:       { include: { matches: true } },
  }
  ```

  Prisma issues this as ~3 SQL queries with deep joins. On a 32-player bracket: 1 tournament + 32 participants joined to users + 5 rounds + 31 matches = a single response Of ~70 rows. Watch for two issues:

  (a) `matches: true` returns *all* TournamentMatch columns including large unused fields (e.g. `metadata` JSONB, `botMoveLog` if present);
  (b) the participant→user join can pull users that are no longer competitively active. *Action:* explicit `select` on every nested include, drop unused columns; benchmark before/after.

- **`listBots()` ELO lookup uses an OR over `(userId, gameId)` pairs.**

  ```
  where: { OR: eloPairs }   // N pairs → N OR branches
  ```

  Postgres can plan this poorly past ~8 branches. Rewrite as a single `WHERE (user_id, game_id) IN (VALUES …)` (raw SQL) or a UNION of exact-match subqueries; or add a composite `(user_id, game_id)` index if not already present. Cheap, measurable.

- **N+1 in spectate / live-state paths.** The bot-game runner emits state events that include only ids; the client backfills with separate fetches per id. Audit `useEventStream` consumers for any pattern that triggers a fetch loop on each event arrival.

- **Audit `select` discipline platform-wide.** Prisma defaults to selecting all scalar fields when `select` is omitted on a nested include. Three places in `listBots` already do this right; verify every `findMany`/`findUnique` in `backend/src/services/*` and `tournament/src/routes/*` follows suit.

F2 — REST response preloading (NEW)

The cold-page Ready cost is dominated by waterfalls: HTML parses → JS parses → bundle queries an API → API returns → render. The plan covers the JS parse but not the *API queries during parse*.

Instrumentation landed (2026-05-04, `dev`): `perf/perf-waterfall.js` measures, per route × device, when the primary `/api/*` call fires (`requestStart` from navStart) and when its response lands (`responseEnd`). Local sanity-check (n=3, desktop, no network throttling) — every route's primary API doesn't fire until ~400ms after navStart, which is pure JS parse + React mount + `useEffect` cost (the API itself is 2–5ms locally):

Route	apiStart	apiTotal	LCP	preload prize (~)
`/`	418ms	5ms	696ms	~400ms
`/tournaments`	377ms	2ms	388ms	~370ms
`/rankings`	385ms	5ms	396ms	~380ms

LCP on `/tournaments` and `/rankings` lands ~11ms after the API returns — preload would shift the entire chain ~370ms earlier. Staging/prod prize is presumably larger (longer bundle download + bigger API total). Re-run with `--target=staging` to confirm before committing implementation effort.

Aside: the waterfall also surfaces ~7 redundant `/api/session` hits per cold page load. Filed as a separate dedupe issue; not part of F2 itself.

- ☐ **`<link rel="preload" as="fetch">`** for the top-1 API call per route, emitted in the HTML `<head>` so the request fires in parallel with bundle download. Routes + targets:
  - `/` → `/api/v1/bots?gameId=xo` (the community bot fetch)
  - `/tournaments` → `/api/tournaments`
  - `/rankings` → `/api/v1/leaderboard?period=all&mode=all&includeBots=false`
  - `/profile` (auth) → `/api/v1/bots?ownerId=…`
- ☐ **103 Early Hints with `Link: rel=preload`** (Section E1) for both static chunks *and* the API preload above. Doubles the effect: preload starts before TTFB *and* parallel to JS parse.
- ☐ **Inline initial payload into `index.html`** for routes whose data shape doesn't depend on auth or query params. Static-ish candidates: built-in bot list, system config keys the client reads first paint. Cuts an entire round-trip on `/`.
- ☐ **HTTP/2 server push fallback?** Skip — browsers have removed it. 103 Early Hints is the modern equivalent and already in E1.
- ☐ **Wire `<Link to>` hover into a fetch warmup** (extending Phase 10): on hover, pre-fire the route's primary API call and cache the response for the upcoming nav.

F3 — Connection pool, transactions, query plans (NEW)

Phase 2 mentions PgBouncer; doesn't audit what's already in place.

- ☐ **Prisma connection pool size.** Default is `num_cpus * 2 + 1`, which on Fly's `shared-cpu-1x` resolves to 3. Under modest concurrency (5+ simultaneous SSE move POSTs) this saturates and queues. Bump explicitly via `connection_limit` query param; benchmark p95 under load before/after.
- ☐ **Transaction footprint audit.** Hot multi-step transactions: bot-game runner per-move, tournament round advancement, training-session completion. Long transactions hold row locks and block concurrent reads. Profile + shrink scope where possible.
- ☐ **Prepared statement reuse.** Prisma's query engine prepares and caches statements per connection — confirm the cache is sized for the actual query variety (default 100; if exceeded, Prisma re-prepares on every call).
- ☐ **`EXPLAIN ANALYZE` the top 5 slow queries** identified in the Phase 2 `pg_stat_statements` pass. A confirmed-slow query plan is what justifies an index, not a heuristic.
- ☐ **`SELECT FOR UPDATE` audit.** `tournamentService` advancement uses row locks; verify the lock window is bounded (no awaiting external service calls inside a `$transaction`).

F4 — Phase 5 (SSE round-trip) decomposition (concrete steps)

The 0.4 data showed POST move → SSE event = 560ms / 670ms p50, with ~400ms unaccounted for between POST ack and event arrival. The single- stream `perf-sse-rtt` decomposition (2026-05-05) put 383ms p50 of that on `publishToPickup` (Redis XADD → broker XREAD wake). The follow-up concurrent load test (`perf/perf-sse-load.js`, 2026-05-05) sharply rewrote what that number represents:

- **It is not Redis throughput** — `pickupToWrite` is < 1ms p50 at every load level, and `apply` (which includes the XADD) is < 20ms p50 even at c=50.
- **It is not load-related** — at c=50 concurrent users the publishToPickup p50 is **21ms** (vs 383ms quiet), p95 106ms.
- **It is the broker's process wake-up cost on a quiet service.** The broker uses `XREAD BLOCK 30_000` (`backend/src/lib/sseBroker.js`: When the backend is idle, the Node process pages out / scheduler deprioritises it / Redis TCP connection's recv-buffer is empty. The first XADD that follows pays a measurable wake-up tax (~150–500ms) before `Date.now()` is sampled in `dispatchEntry`. Once the loop is hot — i.e. any time the broker has dispatched an event in the last few hundred ms — that tax is ~20ms.
- **By move-index** (staging, c=5, n=5 each):
  - move 0 (cold table, may also catch broker idle): **382ms p50, 740ms p95**
  - move 1 (warm channel): 35ms p50, 49ms p95
  - move 2 (warm channel): 55ms p50, 69ms p95

Concurrency sweep — staging, 2026-05-05 `perf/perf-sse-load.js --target=staging --sweep=1,5,10,25` then a follow-up `--concurrency=50`. 3 moves per virtual user, 80ms ramp.

c	n	apply p50/p95	publishToPickup p50/p95	pickupToWrite p50/p95	movePostAck p50/p95	botMoveTotal p50/p95
1	3	7 / 8 ms	55 / 196 ms	0 / 1 ms	166 / 197 ms	257 / 367 ms
5	15	7 / 8 ms	55 / **740 ms**	0 / 0 ms	189 / 260 ms	324 / 932 ms
10	30	7 / 18 ms	32 / 406 ms	0 / 0 ms	195 / 304 ms	278 / 612 ms
25	75	15 / 45 ms	25 / 406 ms	0 / 0 ms	291 / 556 ms	391 / 721 ms
50	150	19 / 41 ms	21 / 80 ms	0 / 0 ms	396 / 604 ms	453 / 633 ms

Reading the table:

- **`publishToPickup` p50 *drops* with load**, from 55 → 21ms. p95 spikes at c=5/10 (740 / 406ms) but is dominated by move-0 cold-wake (see breakdown above). At c=50 the p95 is back to 80ms because the broker loop never goes idle long enough to catch a cold wake-up.
- **`apply` p95 climbs cleanly** (8 → 45 ms = 5.6× at c=25, then 41ms at c=50 — Postgres holds up). This is the only metric where load amplification is monotone and clean.
- **`pickupToWrite` is flat at 0** at every level — broker dispatch loop is not a bottleneck, which kills one Phase 5 sub-task ("audit redis pub/sub round-trip" — answer: it's fine).
- **`movePostAck` p50 ~doubles** by c=50 (166 → 396 ms), driven by apply growth + Node event-loop pressure. This is the dominant user-perceptible regression under load.
- **No fatal failures** at c=50 (50/50 users completed all 3 moves).

This rewrites Phase 5's ROI math and step list. The original in-process-fanout / pubsub-shortcut steps below are still valid, but the headline win is no longer "~300 ms saved on every move": it's "~300 ms saved on the *first* move per fresh table, *only* on quiet services."

- ☒ **Time-trace one move end to end.** *(spike landed 2026-05-04 on `dev`)* — `backend/src/routes/realtime.js` POST `/rt/tables/:slug/move` emits `Server-Timing: lookup;dur=X, apply;dur=Y` (lookup = caller resolution + table findFirst; apply = applyMove + DB writes + Redis XADD). `backend/src/lib/sseBroker.js#dispatchEntry` injects `_t: { publishToPickupMs, pickupToWriteMs }` into the JSON payload of every SSE frame (publishMs derived from the Redis stream id `<ms>-<seq>`). `perf/perf-sse-rtt.js` parses both and reports six decomposition rows: server.lookup, server.apply, network (POST RTT − server), redis publish→pickup, broker pickup→write, network (ack→event − server). **Local sanity-check (n=10):** every leg ≤2ms; total matches client-measured `playerEventMs`. Staging + prod numbers TBD after promote — that's where the ~400ms lives.
- ☒ **Keep the broker hot — kill the cold-wake penalty.** *(Shipped 2026-05-05 in v1.4.0-alpha-4.3 — see Appendix Z.1.7.)* Reduced `XREAD_BLOCK_MS` from 30_000 → 1_000 in `backend/src/lib/sseBroker.js`. Validation on staging: cold first-move publishToPickup p50 dropped **382 → 39 ms** (−90%), p95 **740 → 126 ms** (−83%) at c=5. Warm-move latency unchanged. No alert threshold change needed — resourceCounters' 90 s staleness window is well above 1 s. **Phase 5 ROI no longer includes this win** — remaining Phase 5 steps target the ~270 ms steady-state floor only.
- ☐ **Confirm SSE response is unbuffered.** Express + compression middleware can buffer SSE frames if not disabled per-route. Verify `Content-Encoding: identity` (or no-op compress) on `/api/v1/events/stream` — gzip would buffer the entire stream.
- ☐ **Disable Nagle's algorithm** on the SSE response socket (`socket.setNoDelay(true)`). On Node + Express this is rarely automatic; can shave 40ms per write.
- ☐ **Audit Redis pub/sub round-trip.** SSE channels go through Redis (per `useEventStream`). Time the publish→deliver gap on the same VM. If it's > 50ms, the pubsub topology needs work (sharding, in-process fanout for same-instance subscribers).
- ☐ **Same-instance shortcut.** If the publishing process and the SSE-serving process are the same Fly machine (true for our single-machine setup), skip Redis entirely and dispatch via in-memory event emitter. Redis only kicks in when machines need to fan out to subscribers on other VMs.
- ☐ **Coalesce duplicate events.** A single move can emit `state(moved)` + `lifecycle(...)` + journey progress; if these fire serially with their own SSE writes, latency multiplies. Pack into one frame where possible.
- ☐ Re-measure with `perf-sse-rtt.js`. Target: POST move → SSE event ≤ 200ms p50 staging, ≤ 250ms p50 prod.

F5 — Phase 1 specifics from the visualizer (concrete byte targets)

Phase 1 gives directional sub-steps but no per-chunk byte budget. After running `VISUALIZE=1 npx vite build` once and reading the output, set hard targets for each chunk and fail CI on regression:

- ☐ `vendor-react` ≤ 50 KB gz (current ~82 KB — Preact compat eval)
- ☐ `vendor-charts` (`recharts`) ≤ 25 KB gz (post-uplot swap)
- ☐ `vendor-auth` (Better Auth + better-fetch + zod) ≤ 30 KB gz
- ☐ `main` ≤ 150 KB gz (current 411 KB — biggest cut)
- ☐ Each lazy route chunk ≤ 60 KB gz
- ☐ `game-xo` ≤ 30 KB gz
- ☐ Total first-paint JS ≤ 250 KB gz on Home (current 490 KB)

If any chunk exceeds budget, the PR fails CI without a `perf-ok` label. Re-bake budgets after Phase 1 lands.

F6 — Optimistic move rendering (perceived-perf, NEW)

The 560–670ms POST → SSE round-trip is the ceiling on Phase 5; below that ceiling the user still feels every move's latency unless the client compensates. The existing Phase 12 ("animation budgets") does not address this.

- ☐ **Render the player's move locally the instant the click lands.** Update the local board state, animate the mark, play sound — all before the POST returns. Reconcile when the SSE `state(moved)` event arrives.
- ☐ **Predict the bot's response.** For built-in minimax bots, the same minimax engine ships in the client bundle. After the player's move, run minimax locally to predict the bot's response, render it as a "ghost" preview after ~150ms, then reconcile with the real SSE event. If predictions match (they will most of the time for minimax), latency feels zero.
- ☐ **Disable click during reconciliation window.** Prevents double-tap from creating an inconsistent local state.
- ☐ **Visual cue if reconciliation rolls back.** A subtle shake + color flash when the local prediction was wrong (rare for built-in bots; meaningful for ML bots).

F7 — Streaming + progressive rendering (perceived-perf, NEW)

The plan mentions "streaming responses for tournament detail" inside Phase 9, but doesn't lay out the priority order or the rendering strategy. Expand:

- ☐ **`TournamentDetailPage` priority cascade.** Render the page header (name, status, time) immediately from URL params or the list-cache. Then `<Suspense>` boundaries in this order: bracket → participant table → match history → coaching cards. Each `<Suspense>` resolves independently so the slowest segment doesn't block faster ones.
- ☐ **NDJSON / chunked transfer for large lists.** `GET /api/tournaments/:id` returns a single JSON body today. Switch to NDJSON: header chunk, then participant chunks, then match chunks. Parse and render incrementally on the client.
- ☐ **Stale-while-revalidate as the default for lists.** When the user navigates back to `/tournaments`, show the cached list *instantly* (we already had it in memory), then refresh in the background. Same pattern for `/leaderboard`, `/bots`.
- ☐ **Skeleton density matches content shape.** Cards with avatar circles, table rows with proportional column widths, bracket shapes that mirror the round-N layout. Misshapen skeletons cause layout flicker on resolve and read as worse than no skeleton at all.

F8 — Open measurement gaps (2026-05-05)

After closing the authed-endpoint gap on 2026-05-05, the remaining synthetic blind spots, ranked by likely-to-find-real-issues:

#	Gap	What's missing	Why it matters	Effort
1	**Cold-authed page-level perf**	perf-v2 only does `cold-anon`. We have authed *endpoints* (p95) but not authed *pages* (Ready/FCP/LCP).	Phase 1c is scoped from endpoint math (200→143 ms p50). Actual cold-authed page Ready could be 300 ms slower than cold-anon for *all* signed-in users — we're guessing the win.	1d
2	**TournamentDetailPage**	Not in `perf-v2`'s route list. The 1900-line render path Phase 13 calls out.	Heaviest single surface, most-engaged users, completely unmeasured. Can't tell if Phase 1's lazy-load helps until we benchmark.	1d

#	Gap	What's missing	Why it matters	Effort
3	**Live SSE under realistic load** *(closed)*	~~`perf-sse-rtt` measures **one** move at a time, isolated.~~ Closed 2026-05-05 by `perf/perf-sse-load.js`. Findings rewrote Phase 5's scope — see §F4 above. tl;dr: fan-out load doesn't degrade `publishToPickup`; the 383 ms prod number is **cold-broker-wakeup** on the *first* move into a fresh table, and warm/under-load moves all complete in <60 ms p50.	closed	
4	**Repeat-visit / warm cache** *(closed)*	~~Only cold-anon context.~~ Closed 2026-05-05 by `perf-v2 --context=warm-anon`. Mobile median: Ready **2050 → 370ms (−82%)**, FCP **1424 → 136ms (−90%)**. JS bytes go from 495 KB → 0 (cached). The remaining 370ms warm Ready is parse + execute + index.html revalidation + initial data fetch. **Validates the user's intuition that Phase 1's revisit value is modest** — bytes are already cached on revisit; bundle splitting only chips at the parse residual. SW app shell + SWR data caching are higher-leverage levers for returning users. See §F11.	closed	
5	**DB time as fraction of p95** *(closed)*	~~Endpoint p95 is wall-clock, could be 90% DB or 10% DB.~~ Closed 2026-05-05 by F4 load test + F9 audit. DB itself is fast (apply <20ms p95 at c=50, queries 1-5ms each); pool was hypothesised as the bottleneck but F9.1 deploy showed **no measurable improvement** — the wall-clock ceiling lives in CPU/fsync, not DB or pool. Confirms Phase 2 (DB indexes) stays deferred — see §F9.	closed	
6	**iOS Safari**	Moto G4 (Chromium emulation) is the only mobile signal.	iOS Safari has different scheduling, cache eviction, and SSE handling (6-conn limit per origin is a known footgun). Significant market segment, zero data.	1d
7	**CLS (Cumulative Layout Shift)**	LCP is tracked, CLS is not.	Hero swap, late-loading auth UI, modal reflows all *could* cause shift (budget ≤0.1 per Web Vitals). Probably fine but the not-knowing is itself the gap.	0.5d

Suggested cadence: add one gap per sprint, in rank order. Don't batch — each gap-closing PR is its own measurement → finding cycle.

My recommendation for next gap to close (after Phase 1 lands): gap **#1 (cold-authed pages)**. It directly validates Phase 1c's expected ROI, and we already have the `xo_perf` token plumbing from the `um perfuser` CLI. Gap **#3 (SSE under load)** closed 2026-05-05 — see §F4 above for the full evidence and revised Phase 5 scope.

What stays *out of scope* for the synthetic harness even now: cross-region latency (not actionable from a single test runner), tab-throttling behavior (browser-internal, hard to script), and battery / energy profiling (not a perf-budget conversation). Those go to RUM / production telemetry if they ever become a question.

F9 — DB audit findings (2026-05-05)

Triggered by the F4 load test surfacing `apply p95` growth (8 → 45ms at c=25, 5.6×) — the only metric where load amplification was clean and monotone. Audit covers (a) the immediate `applyMove` regression, (b) F8 gap #5 (DB time as fraction of p95), and (c) other DB risks on hot paths.

F9.1 — Connection pool bumped 10 → 30 ***(shipped 2026-05-05; no measurable gain at this load)*** `packages/db/src/index.js` constructs `new PrismaPg({ connectionString, idleTimeoutMillis: 15_000, connectionTimeoutMillis: 10_000 })`. **No `max` is set, so `pg.Pool` defaults to `max: 10`.** Confirmed by constructing a fresh pool (`new pg.Pool({...}).options.max === 10`).

At c=50 concurrent moves on staging:

- 50 in-flight HTTP requests × 2 queries each (`findUnique` + `update`) = up to 100 query-acquisitions queueing on 10 slots.
- Postgres on Fly.io completes each PK lookup / PK update in ~1–5ms (well-indexed, JSONB writes included).
- Pool wait dominates. Math: 100 acquisitions / 10 slots × 5ms ≈ 50ms per move queue depth. Matches the observed p95 of 41–45ms vs c=1 baseline of 8ms (~33ms additional).

Fix: bump pool to `max: 25–30`. Postgres' default `max_connections` on Fly Postgres is 100+ for small instances, and we have one backend service plus tournament service competing — 30 per backend machine is well within bounds.

```
new PrismaPg({
  connectionString: process.env.DATABASE_URL,
  max: 30,                      // up from default 10
  idleTimeoutMillis: 15_000,
  connectionTimeoutMillis: 10_000,
})
```

Predicted win: `apply p95` at c=25 drops from 45 ms to ~10–15 ms. **Measured (2026-05-05 staging post-deploy v1.4.0-alpha-4.3):**

Concurrency	apply p50 / p95 — before	apply p50 / p95 — after	Δ p95
c=1	7 / 8 ms	11 / 21 ms	mild +
c=5	7 / 8 ms	9 / 29 ms	mild +
c=10	7 / 18 ms	10 / 28 ms	flat
c=25	15 / 45 ms	20 / 46 ms	**flat**

Pool was not the actual bottleneck. Postgres queries each return in 1–5 ms; with `max=10` the pool was rotating fast enough that no significant queue accumulated. The pool bump is a free hedge for higher concurrency (we'd hit the wall at c=100+ on the old setting), but the headline 5.6× p95 growth comes from elsewhere.

Why pool wasn't actually the bottleneck (math):

- Each `applyMove` does 2 Prisma calls (`findUnique` + `update`). Each takes 1-5ms on Postgres.
- At c=25 fully concurrent, peak query load = 50 in-flight.
- With max=10 and 5ms hold time, the pool services `10 × (1000ms / 5ms) = 2000 q/s` capacity.
- Test load: 25 users × ~2 q/move ÷ ~600ms/move = **~80 q/s**.
- We were at **4% of pool capacity** even with max=10. Connection acquisition was never queueing.

Where the cost actually lives (ranked by likelihood):

1. **Backend single shared vCPU** (`backend/fly.toml` — 1 cpu, shared). The strongest candidate. At c=25 Node multiplexes 25 concurrent in-flight requests on one shared core (effectively ~20-30% of a vCPU under contention). All `await` resolutions, JSON encoding, Prisma client SQL generation, and Redis client work share that single thread. Wall-clock per call grows even while the DB itself stays fast. **F9.2 (VM bump to 2 dedicated) is the direct test.** Predicted apply p95 at c=25: 46 → 15-20ms.
2. **Postgres WAL fsync per-commit.** Default `synchronous_commit=on` requires fsync on COMMIT. Fly Postgres on shared storage: typically 5-15ms per fsync. At c=25 burst-write (25 COMMITs queued behind one fsync flush window), batching helps but first-in-batch waits up to a full window. Adds ~10-20ms p95 under burst. Mitigation: `synchronous_commit=off` (durability loss — not acceptable) or batching across moves (architectural change — not worth

it).

3. **JSONB write amplification.** `Table.previewState` is a 2-3 KB JSON blob, fully rewritten each move. Postgres serialises the blob into TOAST, computes the new heap row size, may rewrite the page. Per-update cost is small (~1-3ms) but stacks under write contention.
4. **Prisma client JS-layer overhead.** Even with available connections, Prisma generates SQL, sends, parses result on the same Node event loop. ~1-3ms per call wall-clock just in JS. Multiplied across 25 concurrent loops, contributes to (1).

How to narrow it down — concrete next steps:

- ☒ **Granular Server-Timing inside applyMove.** *(Shipped v1.4.0-alpha-4.4 2026-05-05.)* `applyMove` returns `_t: { findMs, updateMs, postMs }`; realtime.js surfaces `apply.find / apply.update / apply.post` in Server-Timing. **Read on staging 2026-05-05** (full breakdown in Z.1.9): apply.find ≈ apply.update at every load level (5/12 vs 5/15 at c=25, 6/19 vs 7/19 at c=50). apply.post stays ≤2ms p95 everywhere. Verdict: read and write grow proportionally (rules out JSONB-only write contention as the dominant hypothesis); broker dispatch is not the contention. The remaining `movePostAck` growth (163 → 302 ms at c=1 → c=50) lives **outside applyMove** — points squarely at suspect 1 (1 vCPU saturation in Express middleware + handler).
- ☒ **Pool metrics**: surface `pg.Pool totalCount / idleCount / waitingCount` per request. *(Shipped v1.4.0-alpha-4.4 2026-05-05.)* `packages/db/src/index.js` constructs its own `pg.Pool` and passes it to `PrismaPg`; `getPoolStats()` reads counters synchronously; surfaced in Server-Timing. **Read on staging 2026-05-05:** `pool.waiting = 0` at c=1, 5, 10, 25, 50. Peak `pool.total = 9` (max=30, ~30% capacity). Pool was definitively never the bottleneck — F9.1 retroactively confirmed as a no-op for perf.
- ☐ **Direct F9.2 test:** stand up a temporary `cpu_kind = "performance", cpus = 2` Fly machine, point staging traffic at it, re-run the load test. **Now the recommended next lever** — probe results above eliminated DB / pool / broker as suspects. Predicted impact: `movePostAck` p50 at c=50 drops 302 → ~150-180 ms.
- ☐ **Node `--prof` under load** *(only if F9.2 underperforms).* CPU profile while `perf-sse-load --concurrency=25` runs. Look for hot stacks in Prisma serialisation or Express middleware. Defer until F9.2 has been tested — the probe data already implicates the event loop.

Decision: Pool fix stays in (defensive + no cost; would matter at c=100+). Real fix is F9.2 — promoted from hypothesis to recommended next lever based on probe evidence. Probe narrative: Appendix Z.1.9. F9.1 validation numbers: Appendix Z.1.8.

F9.2 — VM sizing as the secondary ceiling `backend/fly.toml`: 1 shared CPU, 512 MB RAM. Even with a larger pool, a single shared vCPU caps how much concurrency the Node event loop can sustain before `movePostAck` p50 climbs (already 166 → 396ms c=1 → c=50). Once F9.1 lands, re-measure: if `movePostAck` is still 2× at c=50 with apply now flat, the remaining cost is Node event-loop pressure on shared CPU.

Cheapest knob: `cpu_kind = "performance"` and/or `cpus = 2`. Cost decision, not a code change. Track separately — not shipped without explicit budget approval.

F9.3 — Closes F8 gap #5 (DB time as fraction of p95) Server-Timing headers on the move POST already split out `lookup` and `apply` — both under 20ms p95 even at c=50. So for the realtime path, **DB time is a ~50% fraction of total movePostAck** (15-20ms server / 30-40ms total minus network). That means a Prisma rewrite or driver swap (Phase 5 driver step) would cap out at saving ~20ms per move — not the dominant win. Confirms Phase 2 (DB indexes) stays "deferred for lack of evidence": apply p95 is already <50ms at c=25; no rewrite needed before pool-tuning lands.

For non-realtime paths (page Ready, /me/* endpoints), the same pattern likely holds — Postgres is fast, but pool wait under concurrency turns wall-clock latency into queueing latency that no index can fix.

F9.4 — Other DB risks (hot-path audit) Findings ranked by likely impact:

#	Concern	Status	Action
1	**`pg.Pool max=10`** (covered in F9.1)	Bottleneck under load	Bump to 25–30
2	`BaSession` has no `userId` index — only `token` indexed	OK as-is	Better Auth's hot path is `token` lookup, which is indexed. `userId`-by-session is admin-panel only — leave
3	`Move` has only `gameId` index	OK as-is	`Move` is append-only; never queried by anything else

#	Concern	Status	Action
4	`Game` indexes are good (outcome, endedAt, players)	OK	Already covers leaderboard + history reads
5	`Table.previewState` is JSONB, full-blob update each move	Acceptable	<5ms per write at staging size; no normalization needed
6	`db.botSkill.findMany`, `db.user.findMany` — no `take`	Bounded by WHERE	Each has tight WHERE clauses (single botId, role filter)
7	`db.log.findMany` paginates with `skip/take`	Good	Already the right pattern

Net: only F9.1 is a real fix. The rest are non-issues, captured here so we don't re-investigate them.

F9.5 — Recommended sequence *(updated 2026-05-05 post-validation)*

1. ~~Ship F9.1 (`max: 30`).~~ **Done** v1.4.0-alpha-4.3. No measurable gain on apply p95 at the load levels tested (c=1 to c=25). Kept as a defensive change for future c=100+ load.
2. ~~Ship F9 probes (granular apply Server-Timing + pool stats).~~ **Done** v1.4.0-alpha-4.4. Probes pinpointed 1-shared-vCPU as the bottleneck and ruled out pool, broker dispatch, and JSONB write contention. Detail in Appendix Z.1.9.
3. **F9.2 (VM bump) — DEFERRED 2026-05-05 on cost grounds.** Probe-validated as the right lever (predicted: movePostAck p50 c=50 drops 302 → ~150-180 ms), but performance-2x VMs add ~+$87/mo across staging + prod (staging ~$31, prod 2 mach ~$62 vs current ~$5.82). Current c≤25 staging perf is acceptable (apply p95 26 ms, movePostAck p50 ~220 ms). Re-promote when prod telemetry shows sustained c≥25, or when Fly's pricing changes. The probes stay wired — when we re-test, the before/after will be clean.
4. Phase 2 (DB indexes) stays deferred — F9.3 confirms DB itself is fast (apply queries < 20 ms p95 even at c=50).

F10 — Sign-in flow latency (2026-05-05)

Triggered by user perception that "Better Auth feels slow." The existing `perf-backend-p95` measured *read* endpoints; sign-in is the write path the user actually feels.

F10.1 — Measurement (extended `perf-backend-p95.js`) Added a fourth measurement tier — auth-flow endpoints — gated on `PERF_AUTH_EMAIL` + `PERF_AUTH_PASSWORD`. POST `/api/auth/sign-in/email` is included with reduced request count (default 50, since each sign-in writes a `BaSession` row).

Staging 2026-05-05 (REQUESTS=100, AUTH_REQUESTS=50, c=5):

Endpoint	p50	p95	p99	ok
GET /api/version	53 ms	188 ms	229 ms	99/100
GET /api/auth/get-session (anon)	55 ms	132 ms	142 ms	100/100
GET /api/auth/get-session (authed)	51 ms	119 ms	141 ms	100/100
GET /api/v1/leaderboard?game=xo	53 ms	101 ms	137 ms	100/100
GET /api/tournaments	67 ms	205 ms	340 ms	100/100
POST /api/auth/sign-in/email	**793 ms**	**1537 ms**	**1659 ms**	50/50

Compare to local (Mac, full CPU): sign-in p50 71 ms. Staging is **~11× slower** — the same shared-1-vCPU contention amplification profile we documented in §F9 / Z.1.9.

F10.2 — Spike: which hash library is in use? `backend/src/lib/auth.js:24-48` overrides Better Auth's default hash with a custom `hashPassword` / `verifyPassword` pair using **`node:crypto.scrypt`** (native binding via libuv thread pool):

```
const SCRYPT_PROD = { N: 16384, r: 16, p: 1 }   // 32 MB memory, ~16.7M ops
const SCRYPT_DEV  = { N: 4096,  r: 8,  p: 1 }   // 4 MB memory
const KEYLEN      = 64
password: { hash: hashPassword, verify: verifyPassword }
```

Better Auth's *default* is `@noble/hashes/scrypt` — pure JS, runs on the V8 main thread. XO already swapped that out, so:

- ☐ Hash is on libuv's thread pool (4 threads default), not main
- ☐ Worker threads would NOT add value — work is already off-main
- ☐ `SCRYPT_PROD` is at OWASP 2024 minimum; cutting `r=16 → r=8` drops below the floor

F10.3 — Decision Reframed lever ranking after the spike:

#	Lever	Verdict
1	~~Lower scrypt rounds~~	Already at OWASP minimum; not pursuing without security review
2	F9.2 (VM bump 1 → 2 dedicated CPUs)	Right fix; deferred on cost
3	~~Worker threads~~	Dead — already on libuv pool
4	**Async UX** (progressive sign-in feedback)	**The right move** — ~30 LOC in SignInModal
5	Rate limit	Protects backend, doesn't help user wait

The actual sign-in latency (~800 ms p50) is fundamentally bound by memory-hard scrypt × 1 shared vCPU. Without F9.2 or weakening the hash, no plumbing change makes a single sign-in faster. The remaining lever is **perceived-latency UX work**: progressive spinner + helper text in SignInModal so the wait feels intentional rather than stuck.

SignInModal shipped 2026-05-05 in commit `954f353`: inline spinner SVG + delayed helper text ("Verifying your password…", "Creating your account…", "Sending reset link…") after 400 ms. 9/9 tests pass.

F10.4 — Other Better Auth surfaces (not yet reviewed) Same scrypt-on-1-vCPU latency profile applies wherever the backend hashes or verifies a password. Surfaces that *probably* have the same dead-air UX gap, ranked by likely impact:

Surface	Backend wait	Reviewed?	Priority
`landing/src/pages/SettingsPage.jsx` — change password	~800 ms (verify + hash)	☐	medium
`landing/src/pages/ResetPasswordPage.jsx` — submit new password	~800 ms (hash)	☐	**high** — failure here looks like the reset flow is broken
`GoogleSignInButton` / `AppleSignInButton` — OAuth click→redirect	browser handles nav feedback	☐	low — probably fine
`EmailVerifyBanner` resend button	~50-100 ms (no hash)	☐	low — rarely used
`signOut` (AppLayout / ProfilePage / SupportPage)	~50 ms (session invalidate)	☐	skip — fast enough

Effort to apply the same spinner+helper pattern: ~10-20 min per surface. Could extract `Spinner` + `useProgressHelper` into `landing/src/lib/loadingFeedback.jsx` if we touch ≥2 more files. Filed as Tier 1 perceived-perf follow-up.

F11 — Warm-cache (returning-user) baseline (2026-05-05)

Closes F8 gap #4. The single most important measurement we never had: what does a returning user actually experience? Every prior benchmark in the trend was cold-anon — fresh browser context, no cache. Returning-user perf was assumed but unmeasured.

F11.1 — Implementation `perf-v2.js` gained a new `warm-anon` context. Same as `cold-anon` except a single `browserContext` is reused across the route's runs so the HTTP cache survives. Each route runs RUNS+1 hits and discards the first (priming, cold) — p50/p95 reflect the warm-cache reality of a user reloading or returning between deploys.

Throttling stays per-page via CDP, so the same network profile applies to every visit including the priming hit. Commit `12f1322`.

F11.2 — Findings (staging, 2026-05-05, n=5/route) Mobile (Moto G4 / 4G):

Route	Cold Ready p50	Warm Ready p50	Δ	Cold FCP	Warm FCP	Δ
Home	1760 ms	**357 ms**	−80%	408 ms	**68 ms**	−83%
Play	2062 ms	**371 ms**	−82%	1404 ms	**136 ms**	−90%
Leaderboard	2070 ms	**365 ms**	−82%	1424 ms	**132 ms**	−91%
Tournaments	2065 ms	**349 ms**	−83%	1420 ms	**140 ms**	−90%
Tables	2077 ms	**428 ms**	−79%	1448 ms	**148 ms**	−90%
Stats	2048 ms	**431 ms**	−79%	1404 ms	**188 ms**	−87%
Median	**~2050 ms**	**~370 ms**	**−82%**	**~1424 ms**	**~136 ms**	**−90%**

Desktop:

Route	Cold Ready p50	Warm Ready p50	Δ
Home	878 ms	**401 ms**	−54%
Play	849 ms	**376 ms**	−56%
Leaderboard	811 ms	**380 ms**	−53%
Tournaments	767 ms	**313 ms**	−59%
Tables	883 ms	**456 ms**	−48%
Median	**~840 ms**	**~400 ms**	**−52%**

`PlayVsBot` is the outlier — warm Ready stays ~1000ms even with cache. The sequential-init chain (`getCommunityBot()` → `/api/v1/rt/tables` POST → redirect) doesn't benefit from HTTP asset caching. Filed as Phase 1b — already documented.

JS bytes go from 495 KB cold → "—" warm (Resource Timing reports 0 transfer when assets are cache-served). Wire bytes on revisit are essentially zero.

F11.3 — What's left in the warm 370ms (mobile) / 400ms (desktop) After cache eliminates download + decode + most parse work, what remains:

1. **`index.html` revalidation** — small (~5 KB) but still a RTT. Browser sends `If-None-Match`, server returns `304 Not Modified` or new HTML if we deployed. ~50-150ms on mobile 4G.
2. **Bundle parse + execute** — V8 has a bytecode cache but it's not always reused (different on tab restart, low-memory mobile). Even a re-parse of the cached bundle takes ~100-300ms on Moto G4. This is what Phase 1 (per-route splits) would chip at — by parsing only the route's slice, not the whole bundle.
3. **Initial data fetch** — `useOptimisticSession` calls `/api/session` on cold mount. Authed users add `users.sync` + page-specific endpoints. ~50-200ms before page is "ready" by our spinner-detached definition.
4. **Initial paint + LCP work** — laying out the (now-instant) hero, loading any uncached images, layout pass. Small (~30-50ms).

Phase 1 would impact #2 only. Best-case mobile warm Ready after Phase 1: maybe ~250-300ms (vs ~370ms today). Real but modest.

F11.4 — Roadmap revision The user's intuition was correct: **for returning users, Phase 1 is a modest marginal win.** The headline returning-user levers, ranked by impact:

1. **Service Worker app shell** (Phase 20) — eliminates even the index.html revalidation + provides offline. Repeat Ready approaches paint-only cost (~50-100ms). The single highest- leverage perceived-perf change in the plan for the engaged-user cohort. **Filed for Tier 1 promotion.**
2. **SWR data caching** + **hover-intent prefetch** — for data- heavy pages (Tables, Tournaments), even the 200ms data fetch becomes "instant render of stale + background revalidate." Combined with #1, returning-user pages feel native-fast.
3. **Phase 1 (bundle splits)** — modest revisit win (~370 → ~250ms mobile parse), still **the right call for first-time visitors** (cold-anon mobile FCP 1416 → ~700ms is the headline). Ships for that audience.

Cohort-aware framing for Tier 0:

- **First-time / cold-anon users:** Phase 1 wins. Phase 3 (image diet) □ + Phase 1d (instant hero) □ already help.
- **Returning users:** SW + SWR + hover-prefetch wins. Phase 1 helps a little.

We don't yet know the cohort split. D1 RUM is shipped but we haven't analyzed engaged-user vs new-user numbers. F11.5 below.

F11.5 — Open question: cohort split in production Until we segment RUM data by returning-vs-new, we can't say which cohort dominates. The work required:

- Use the existing RUM beacon's cookie-set timestamp to segment `cold-first-visit` vs `repeat-visit` in the admin Web Vitals dashboard.
- 1-2 hours of admin panel work + a SQL `GROUP BY` on a synthetic "first-visit" flag.

Filed as Phase 0.2 follow-up. Until we have that data, treat Phase 1 (cold-anon win) and SW + SWR (returning-user win) as **both Tier 0** with sequencing decided by which user pain we hit first in support / dogfood.

Appendix Z — Completed measurement work (archive)

This appendix collects detailed findings from completed instrumentation and measurement passes. The summaries that *inform current decisions* stay in the active sections above; the verbose detail and historical tables live here so the working portion of the plan stays readable.

Z.1 — Targeted gap measurements (2026-05-04 / 05)

The 0.1 baseline (2026-05-02) left four open questions. New scripts plugged each one without waiting on RUM (0.2) or Prometheus (0.3):

- **`perf/perf-inp.js`** — INP per page via `PerformanceObserver({type: 'event', durationThreshold: 16})`, 5 runs × desktop+mobile.
- **`perf/perf-sse-rtt.js`** — SSE connect, POST move ack, POST move → SSE state event, POST move → bot move event, 20 runs.
- **`perf/perf-backend-p95.js`** — concurrent loader, 200 reqs × concurrency 5 across the five hot read endpoints.
- **`perf-v2.js --extended-resources`** — wait 5s post-Ready and re-collect resource bytes, exposing late-loading images that `transferSize` reports as 0 at Ready time.

Saved at `perf/baselines/{inp,sse-rtt,backend-p95,perf}-{env}-*.json`.

Z.1.1 — INP — the click-to-paint floor is excellent Where the script could find a stable interaction, every measured route sits **well under the 200ms "good" threshold** on both desktop and Moto G4 mobile:

Interaction	Desktop p50	Desktop p95	Mobile p50	Mobile p95
Home — refresh demo	**24ms**	120ms	24ms	32ms
Home — open sign-in	**32ms**	32ms	24ms	40ms
Puzzles — first button	**24ms**	24ms	24ms	24ms

Two routes (Tournaments filter, Leaderboard "Show bots" toggle) hit selector-miss issues during the run; the data we have already says **INP is not a problem on the platform today** for the routes that landed. Phase 15 (Guide INP audit) drops in priority — the *idle* floor is fine.

Z.1.2 — SSE round-trip — perceived-latency deep dive Every PvP / PvB move passes through this path. Headline numbers from the 2026-05-05 baseline (post Fly-Replay fix):

Phase	Staging p50	Staging p95	Prod p50	Prod p95
SSE connect → session	194ms	258ms	179ms	263ms
POST move → ack	173ms	233ms	185ms	252ms
POST move → SSE state	**656ms**	902ms	**577ms**	926ms
POST move → bot move event	657ms	902ms	578ms	926ms

Three load-bearing facts that informed the Tier 0 ranking:

1. **The POST acks at 174–262ms but the SSE event arrives 386–408ms later.** That gap is the SSE pub/sub dispatch — flushing the channel write through to the same client's open EventSource. *This is the perceived-perf bottleneck on every move.*
2. **Bot move and player move arrive ~simultaneously** because the backend dispatches both state events in the same request handler.
3. **Prod is ~120ms slower than staging on the SSE round-trip.** Same machine class, same code. Likely cold-machine flap.

Z.1.3 — Multi-machine Fly-Replay fix (2026-05-04) The prod baselines collected on 2026-05-04 surfaced a real production bug, not a measurement artifact. The first prod run after the iad migration showed **18/20 SSE round-trips failing** with `409 SSE_SESSION_EXPIRED`; staging was 0/20.

Root cause: prod runs **2 backend machines** behind Fly's round-robin load balancer. The SSE session registry is a per-process `Map` keyed by session id, so any `/rt/*` POST that hit the *other* machine looked up an unknown session and 409'd. Staging (1 machine) never tripped it.

Fix (commit `0771718`): SSE session ids are now minted with a machine-id prefix (`<FLY_MACHINE_ID>.<nanoid>`). The `/rt/*` middleware decodes the prefix and, if it doesn't match the current machine, returns the `Fly-Replay` header to retry the request on the owning machine. ~30 LOC primitive in `backend/src/realtime/flyReplay.js` plus 2 call sites.

Post-fix prod baseline: **0/20 failures, 20/20 valid samples.** Code- path overhead unchanged (`server.lookup` 4ms, `server.apply` 8ms).

Baseline discontinuity: `perf/baselines/sse-rtt-prod-2026-05-04T23-53-58-761Z.json` and earlier prod F4 baselines were computed over the 2 lucky runs that landed on the SSE-owning machine — their p50/p95s look artificially low. Use `perf/baselines/sse-rtt-prod-2026-05-05T01-17-08-425Z.json` (the first post-fix run) as the new prod F4 reference.

A future Redis-backed session registry (which would replace Fly-Replay entirely and enable non-Fly hosting) is captured in `doc/Future_Ideas.md` and is deferred until non-Fly hosting is on the table.

Z.1.4 — Prod re-baseline 2026-05-05 (full suite, post Fly-Replay) `perf/perf-rebaseline.sh prod` ran all 7 scripts against v1.4.0-alpha-4.0 in 555 s. **Headline takeaways:**

- **Backend latency, all green.** No endpoint p95 over 140ms; only flag is the Better Auth rate limit on synthetic `get-session`.
- **SSE 0/20 failures.** The Fly-Replay fix is doing its job in prod.
- **Cold-page Ready** — desktop p50 ~750–990 ms; mobile (Moto G4 / 4G) p50 in a tight ~2030–2080 ms band. Mobile dominated by JS parse (496 KB bundle) + the (then-) 888 KB hero image.
- **TBT** — desktop = 0 ms; mobile Home p50 = 99 ms (p95 121 ms). All TBT lives in mobile JS parse, not application code.
- **INP** — every measured interaction p50 ≤ 32 ms on both devices.

Two items the new baselines re-confirmed as load-bearing (now Tier 0): the 888 KB hero image on every cold-anon page (Phase 3 — shipped) and POST move → SSE state ~577 ms p50 prod (Phase 5 — open).

Z.1.5 — Backend endpoint p95 (2026-05-05)

Endpoint	Stage p50	Stage p95	Stage p99	Prod p50	Prod p95	Prod p99
`GET /api/version`	52ms	135ms	225ms	54ms	139ms	158ms
`GET /api/v1/bots?gameId=xo`	72ms	184ms	305ms	68ms	140ms	157ms
`GET /api/v1/leaderboard?game=xo`	52ms	113ms	137ms	53ms	120ms	144ms
`GET /api/auth/get-session`	57ms	139ms	147ms	57ms	135ms	206ms
`GET /api/tournaments`	67ms	143ms	256ms	54ms	137ms	160ms

All endpoints 200/200 ok except prod `get-session` at 90/200 (Better Auth's default rate limiter trips at concurrency 5; real users won't hit it, the synthetic harness needs a whitelist — filed as a Tier 1 follow-up).

Z.1.6 — Hero image: candidates evaluated + shipped (Phase 3) **Extended-resource capture** confirmed `colosseum-bg.jpg` was 909 KB on every measured route, starting around 500–700ms (overlapping the Ready window). Original perf-v2 polled `transferSize` while the resource was still in flight (Resource Timing reports 0 until body fully lands).

Candidates evaluated 2026-05-05 (in `perf/hero-candidates/`):

Candidate	Spec	Size	vs original
Original	1920×1279 JPEG	888KB	—
A	1600w WebP q70	174KB	**−80%**
B	1280w WebP q55 (browser scales)	94KB	**−89%**
C	960w WebP q60 + sharpness 7 blur	64KB	**−93%**
D	800w WebP q50 + sharpness 7 blur	47KB	**−95%**

Because the image renders at `--photo-opacity: 0.18` (light) / `0.06` (dark), encode-time blur and aggressive downscale are imperceptible — the eye sees a tinted wash, not a photograph.

Shipped 2026-05-05 (commits `28b7aca` + `f14d052` /stage to staging v1.4.0-alpha-4.1) — went with a 2-asset responsive setup via CSS `@media (min-width: 768px)`:

Tier	File	Size	Spec
Mobile	`colosseum-mobile.webp`	50KB	800w q55 sharpness 7
Tablet+	`colosseum-desktop.webp`	174KB	1600w q70

Original `colosseum-bg.jpg` (888 KB) kept in `/landing/public/` as a silent fallback.

Measured impact (staging v1.4.0-alpha-4.1, 2026-05-05 baseline):

Metric (Home, cold-anon)	Before	After (staging)	Delta
`img_kb` mobile	888 KB	**50 KB**	**−94%**
`img_kb` desktop	888 KB	**174 KB**	**−80%**
Mobile TBT p50	62 ms	50 ms	−19%
Mobile LCP p50	1816 ms	1792 ms	−1%
Mobile Ready p50	2052 ms	2041 ms	−1%

Ready/LCP movement on mobile is small because mobile cold-anon is JS-parse-bound on Moto G4 (Phase 1 territory); the WebP win lands primarily on **bytes-over-the-wire** (~838 KB saved per cold mobile visit on 4G).

Z.1.7 — Cold-broker XREAD fix (Phase 5.1, 2026-05-05) **Hypothesis tested:** the 383 ms publishToPickup p50 we'd been attributing to Redis fanout is actually broker process wake-up cost on a quiet service — under load (broker constantly busy) it should drop, on cold first-moves into a fresh table it should spike.

Evidence (`perf/perf-sse-load.js` 2026-05-05 staging, before fix):

c	n	publishToPickup p50/p95	apply p50/p95	movePostAck p50/p95
1	3	55 / 196 ms	7 / 8 ms	166 / 197 ms
5	15	55 / **740 ms**	7 / 8 ms	189 / 260 ms
10	30	32 / 406 ms	7 / 18 ms	195 / 304 ms
25	75	25 / 406 ms	15 / 45 ms	291 / 556 ms
50	150	21 / 80 ms	19 / 41 ms	396 / 604 ms

The signature: cold first-move (move 0 of each game) p95 was **740 ms at c=5 / 676 ms at c=25**, while warm moves (move 1+) were < 70 ms regardless of load. At c=50 even move 0 was fast (p95 106 ms) because the broker never goes idle.

Diagnosis: `backend/src/lib/sseBroker.js` used `XREAD BLOCK 30_000`. When the backend is idle, the Node process pages out / scheduler deprioritises it / Redis TCP recv buffer empty. The first XADD that follows pays a 150–500 ms scheduling tax before `Date.now()` is sampled in `dispatchEntry`.

Fix: `XREAD_BLOCK_MS = 30_000 → 1_000`. The broker loop now iterates every second even when idle. resourceCounters' 90 s staleness threshold is well above 1 s; no alert wiring change. Code: `0dcb244`. Cost: ~30 idle XREAD/min vs 1; trivial.

Validation (after deploy v1.4.0-alpha-4.3, sweep 1/5/10/25):

c	publishToPickup p50/p95 — before	publishToPickup p50/p95 — after	p95 Δ
1	55 / 196 ms	97 / 137 ms	**−30%**
5	55 / **740 ms**	23 / **126 ms**	**−83%**
10	32 / 406 ms	17 / 46 ms	**−89%**
25	25 / 406 ms	31 / 77 ms	**−81%**

Cold-first-move at c=5: p50 **382 → 39 ms (−90%)**, p95 **740 → 126 ms (−83%)**. The wake-up tax is gone.

Single-stream `perf-sse-rtt` (n=20) confirms the same win on the isolated path:

Metric	Before (2026-05-05 06:30 UTC)	After (post-deploy)	Δ
publishToPickup p50	526 ms	113 ms	−78%
publishToPickup p95	635 ms	191 ms	−70%
movePostAck p50	200 ms	184 ms	−8%

Phase 5 ROI rewrite: The "300 ms saved on every move" pitch is downgraded to "300 ms saved on the first move per fresh table, *only* on quiet services" — and that's now shipped. Remaining Phase 5 steps target the ~270 ms steady-state floor only: in-process fanout for same-instance subscribers, Nagle disable, event coalescing.

Z.1.8 — DB connection pool fix (F9.1, 2026-05-05) **Hypothesis tested:** the apply p95 5.6× growth at c=25 (8 → 45 ms) was pg.Pool default `max=10` saturating under c=25 × 2 queries per move = ~50 acquisitions on 10 slots.

Fix: `packages/db/src/index.js` — added `max: 30` to `PrismaPg({...})`. Code: `3280859`.

Validation (after deploy v1.4.0-alpha-4.3):

c	apply p50 / p95 — before	apply p50 / p95 — after	Δ p95
1	7 / 8 ms	11 / 21 ms	mild +
5	7 / 8 ms	9 / 29 ms	mild +
10	7 / 18 ms	10 / 28 ms	flat
25	**15 / 45 ms**	**20 / 46 ms**	**flat**

Result: no measurable improvement. Pool was not the actual bottleneck. Postgres queries each return in 1–5 ms; the pool of 10 was rotating fast enough that no significant queue accumulated at c=25. The fix is kept in (free hedge for higher loads — would matter at c=100+) but the headline cost lives elsewhere.

Revised root-cause hypothesis (untested, but two suspects):

1. **1 shared vCPU on the backend.** At c=25 the Node event loop contends with itself; Prisma serialization, Express middleware, JSON encode/decode all share one CPU. Manifests as wall-clock on every Prisma call. Addressable via F9.2 (VM bump).
2. **Postgres WAL fsync per-commit serialization.** 25 concurrent `Table.update` commits each fsync; Postgres serialises commit waits. Fundamental Postgres property; addressable only by `synchronous_commit=off` (durability cost) or batching.

Decision recorded in F9.5: F9.2 (VM bump) is the next lever, pending budget approval.

Z.1.9 — F9 probe sweep: pool / apply-band evidence (2026-05-05) **Hypothesis tested:** after F9.1's pool bump showed no apply p95 gain, three suspects remained — (1) 1 shared vCPU, (2) WAL fsync / JSONB write contention, (3) broker dispatch + emit. The probes were designed to discriminate.

Probes shipped (commit `7812210`, deployed v1.4.0-alpha-4.4):

- `applyMove` returns `_t: { findMs, updateMs, postMs }` — wraps `db.table.findUnique`, `db.table.update`, and the broker dispatch + io.emit + appendToStream tail respectively.
- `getPoolStats()` reads `pg.Pool totalCount / idleCount / waitingCount` synchronously; surfaced per-request in Server-Timing.

Sweep on staging 2026-05-05 (`perf-sse-load --target=staging --sweep=1,5,10,25,50 --moves=3 --ramp=80`):

c	apply p50/p95	find p50/p95	update p50/p95	post p50/p95	pool.total p50/p95	pool.waiting
1	8 / 10	3 / 3	4 / 5	0 / 0	1 / 1	**0 / 0**
5	6 / 27	3 / 21	3 / 6	0 / 2	2 / 2	**0 / 0**
10	8 / 26	3 / 11	4 / 13	0 / 1	4 / 4	**0 / 0**
25	10 / 26	5 / 15	5 / 12	0 / 1	6 / 6	**0 / 0**
50	15 / 36	6 / 20	7 / 19	0 / 1	8 / 9	**0 / 0**

Findings (verdict on each suspect):

1. **Pool — RULED OUT.** `pool.waiting = 0` at every concurrency level. Peak `pool.total = 9` against a max of 30 — ~30% capacity. F9.1 retroactively confirmed as a no-op for perf (kept defensively for c≥100 future).
2. **JSONB write contention (WAL fsync) — WEAKER THAN EXPECTED.** `apply.update` p95 grows 5 → 19 ms across c=1 → c=50, but `apply.find` p95 grows the same way (3 → 20 ms) — and `apply.find` is a simple PK SELECT, no fsync, no WAL. Both bands track each other proportionally. If WAL fsync were the dominant contention, `update` would diverge sharply from `find`; it doesn't. Both are growing because they share the same Node event loop.
3. **Broker dispatch — RULED OUT.** `apply.post` p95 ≤ 2 ms at every level. The post tail (helpers import + io.emit + Redis XADD kick-off) is not a contention point. **Phase 5 in-process fanout step can be deferred** with high confidence — it would save ≤ 2 ms per move.

The remaining 290 ms of `movePostAck` p50 growth (163 → 302 ms, c=1 → c=50) lives **outside `applyMove`**: Express middleware, auth, `lookupTableForCaller`, JSON parse, network. That growth profile — wall-clock latency on every step that touches Node — **is the 1-shared-vCPU saturation signature.** All concurrent handlers share one core; under contention, every async resolution, JSON encode, and middleware hop pays the queue.

Roadmap impact:

- **F9.2 (VM bump 1 shared → 2 dedicated) promoted from Tier 1 to Tier 0 rank 2** as the recommended next perf lever, with hard probe data behind it.
- **Phase 5 remaining steady-state work demoted from Tier 0 to Tier 1** — the steps target ≤ 21 ms p50 of remaining publish hop; modest, and only worth it if F9.2 underwhelms.
- **Pool fix stays in** — defensive hedge for future c≥100 loads.
- **Phase 2 (DB indexes) stays deferred** — DB itself is fast; query growth is event-loop pressure, not query plans.

Headline win predicted from F9.2: `movePostAck` p50 at c=50 drops 302 → ~150-180 ms (validate by re-running the same sweep on a 2-CPU machine).

Out of scope (and why)

Option	Reason skipped
SSR / Next.js	Major rewrite. Current SPA already hits FCP ~130ms; SSR's wins (LCP) are addressable cheaper via Phase 3 (image) + Phase 1 (bundle). Revisit only if mobile LCP > 700ms after Phases 1+3.
CDN / Cloudflare in front of Fly	Discussed in v1 — still ruled out for cost / ops; Fly's anycast already gets us most of the way. Revisit when international traffic is non-trivial.

Option	Reason skipped
Redis / external cache	Same call as v1 — in-process cache + per-service warmth is enough. SSE pubsub already uses Redis.
Drizzle / Prisma rewrite	Same call as v1 — query rewrite cost > expected gain. Phase 5 (driver) already in.
Move ML training to backend	Tracked separately in `doc/Connect4_Ship_Checklist.md` Spike B — perf is one input but architectural decision is broader.
Tensorflow.js conversion	Same — `doc/Connect4_Ship_Checklist.md` Spike A.

Sequencing *(updated 2026-05-02 from snapshot data)*

The original sequencing was a guess; the snapshot data lets us tier the phases by expected impact. **Tier 0 must land before Tier 1 even gets a benchmark slot** — otherwise the bundle floor masks any other win.

Tier 0 — sorted by combined real + perceived benefit (2026-05-05)

Ranking criteria: **(1) ms saved per request × frequency × user reach = total real impact**, **(2) user-visible delta on the click-to-feel floor = perceived impact**. Sequencing constraints noted inline where a later phase depends on an earlier one shipping first.

Rank	Phase	Real ms benefit	Perceived	Effort	Status
—	3	mobile img_kb 888→50, desktop 888→174 (−94/−80%)	low/mid	1d	□ shipped v1.4.0-alpha-4.1
—	1d	mobile FCP 1416→432ms (−69%), LCP 1792→432ms (−76%)	**huge**	1d	□ shipped v1.4.0-alpha-4.2
—	5.1	publishToPickup p95 740→126ms at c=5 (−83%); cold first-move p50 382→39ms (−90%)	**huge**	1h	□ shipped v1.4.0-alpha-4.3
—	F9.1	pg.Pool max 10→30 — no measurable gain at c≤25 (free hedge for c≥100)	none	1h	□ shipped v1.4.0-alpha-4.3 (ineffective)
1	1	mobile FCP 1400→~700ms (−700ms × every cold visit)	**huge**	3-5d	next
2	5	remaining steady-state SSE work (~270ms floor)	mid	3-5d	sequenced after 1
3	1c	cold-authed Ready 200→143ms p50 (−30%)	mid	0.5d	queued
4	17	locks Phase-1 gains; direct ms = 0	meta	0.5d	sequenced after 1

Completed (this Tier 0 round):

- □ **Phase 3** — Hero image diet. **Shipped v1.4.0-alpha-4.1 (2026-05-05).** Responsive WebP via CSS media query: 50 KB mobile / 174 KB desktop (−94/−80% from 888 KB JPG). Mobile TBT 62→50ms. Detail in Appendix Z.1.6.
- □ **Phase 1d** — Instant-ready hero. **Shipped v1.4.0-alpha-4.2 (2026-05-05)** in commit `d5cfabd`. Inline static board markup in `landing/index.html`; React `createRoot` swap on hydrate. Mobile FCP **1416→432 ms (−69%)**, LCP **1792→432 ms (−76%)**, desktop FCP **804→424 ms (−47%)**. Ready unchanged (bundle parse still dominates) but the *visible* "page is alive" moment now lands at the HTML-render boundary — biggest perceived-perf single move in the

plan, delivered for 1 day's work.

- **Phase 5.1** — Cold-broker XREAD fix. **Shipped v1.4.0-alpha-4.3 (2026-05-05)** in commit `0dcb244`. Reduced `XREAD_BLOCK_MS` from 30s → 1s in `backend/src/lib/sseBroker.js`. publishToPickup p95 at c=5 **740 → 126 ms (−83%)**; cold first-move p50 **382 → 39 ms (−90%)**. Phase 5's headline ROI was this fix; remaining Phase 5 steps target the ~270 ms steady-state floor only. Detail in Appendix Z.1.7.
- **F9.1** — pg.Pool max 10 → 30. **Shipped v1.4.0-alpha-4.3** in commit `3280859` but **had no measurable gain at c≤25**. Pool was not the actual bottleneck; queries are 1–5 ms each and the pool was rotating fast enough to never queue. Kept in as a free hedge for c≥100. Real apply p95 ceiling is elsewhere (1 shared vCPU
  - WAL fsync are leading suspects — see §F9). Detail in Appendix Z.1.8.

Active queue (sorted by combined real + perceived benefit):

1. **Phase 1** — Bundle audit + per-route splitting. **The single biggest remaining Tier 0 lever** — affects every cold visitor on every page. Concrete sub-steps come from the visualizer; expected to drop `main` from 411 KB gz to ≤ 200 KB gz and mobile FCP from ~1400 ms to ~700 ms (mobile Ready is a flat ~2050 ms band — JS parse dominates). Real benefit ≈ 700 ms × every cold-anon and cold-authed visit; perceived benefit huge (FCP is the most visible user metric).

2. **Phase 5** — Remaining steady-state SSE work. *Scope narrowed after Phase 5.1 shipped.* Cold-wake spike eliminated; remaining floor is ~270 ms p50 of which `publishToPickup` is now 113 ms p50 / 191 ms p95 (single-stream, post-fix). Cutting it further means moving pub/sub closer (Redis on Fly proper, in-region replica) or collapsing the event hop entirely (write directly to the originating connection's response, skipping pub/sub for the single-machine fast path). Concrete steps in §F4: SSE-unbuffered confirm, Nagle disable, in-process fanout, event coalesce. **Sequenced after Phase 1** — re-measure first; bundle work likely shrinks the perceived gap.

3. **Phase 1c** — Cold-authed orchestration cleanup. Memoize `api.users.sync` with a per-token in-flight `Map<token,Promise>` so the two parallel effects in `AppLayout.jsx` (`:288` and `:439`) share one round-trip. Critical path drops from ~200 ms p50 / 262 ms p95 to ~143 ms p50 / 205 ms p95 — a **30% cut on every cold-authed first paint**. ~30 LOC. Half-day. *Could land in parallel with Phase 1.*

4. **Phase 17** — CI bundle-size guard. Fail PR on > 5% chunk growth. *Sequenced after Phase 1.* Direct ms = 0; meta-value high (locks in Phase 1's gains so future feature work doesn't re-bloat them). Half-day to wire.

(**Phase 1b — PlayVsBot deep-dive** dropped from Tier 0 on 2026-05-05. The latest baseline shows PlayVsBot Ready at 991 ms desktop / 2058 ms mobile — same band as every other route. The "uniquely slow" rationale is gone. Moved to Tier 2; will be re-promoted only if post-Phase-1 measurements show PlayVsBot specifically lagging.)

Measurement gaps to close before declaring Tier 0 "done" Today we baseline cold-anon synthetic only. Before claiming the Tier 0 floor is real, two gaps need to close:

- **Authenticated-route p95** — *Measured 2026-05-05 staging.* Wired via `um perfuser` (CLI command that creates a synthetic test user and mints a Better Auth JWT) + `PERF_AUTH_TOKEN` in `perf-backend-p95.js`. **All 6 authed endpoints under the 200 ms p95 budget**: `/users/me/roles` 142ms, `/notifications` 119ms, `/preferences` 137ms, `/hints` 127ms, `/bots/mine` 157ms, `/guide/preferences` 135ms. Authed dispatch adds ~10ms p50 over anon — negligible.

  Client orchestration audit (2026-05-05). Reviewed the actual cold-authed critical path in `landing/src/components/layout/AppLay`

  - `landing/src/store/guideStore.js`. The fan-out is *less wasteful than feared* (only 2-3 endpoints, not 6 — `/me/roles`, `/me/preferences`, `/bots/mine` are page-specific not on landing) but still has avoidable serialization:

#	Issue	Location	Impact (p50)
1	`api.users.sync` called **twice** in two parallel effects	`AppLayout.jsx:288` and `:439`	+70ms wasted RTT
2	Effect 1 serializes sync → `guide/preferences`	`AppLayout.jsx:288→296` (`hydrate()`)	unavoidable (hydrate needs User row)
3	Effect 2 serializes sync → `users/me/notifications`	`AppLayout.jsx:439→441`	could parallelize *if* sync is dedupe'd

#	Issue	Location	Impact (p50)
4	`api.users.sync` is not memoized — no in-flight promise share	`landing/src/lib/api.js:139`	ground-truth for #1 / #3

Critical path today: max(effect 1, effect 2) ≈ **200 ms p50 / 262 ms p95**.

- Effect 1: sync (70ms) → guide/prefs (64ms) = 134-205ms serial
- Effect 2: sync (70ms) → notifications (73ms) = 143-262ms serial, runs in parallel to Effect 1.

With sync deduplication (memoize the in-flight `users/sync` promise so both effects share one round-trip): critical path drops to ~143 ms p50 / ~205 ms p95 — a **~60 ms / 30% cut** on every cold-authed first paint. Single-PR fix in `landing/src/lib/api.js` (wrap `users.sync` in a per-token in-flight `Map<token,Promise>` pattern; clear the entry on settle so retries still work).

Filed as **Phase 1c — cold-authed orchestration cleanup** in Tier 0 (promoted from Tier 1 — cheap, can ship in parallel with Phase 1).

- **DB time as a fraction of endpoint p95** — Phase 2 is currently deferred for "lack of evidence", but endpoint p95 is wall-clock and could be 90% DB or 10% DB; we can't tell. Wire OpenTelemetry or pino-trace into the endpoint hot paths (Phase 0.3) to settle the Phase 2 stay-or-promote question definitively.

Tier 1 — sorted by combined real + perceived benefit

After Tier 0, **re-run `perf-v2.js`** and re-measure. The remaining gap between the new numbers and the budgets table determines what stays. Sort below is best-current-estimate; promote / demote as the post-Phase-1 numbers come in.

Rank	Phase	Real ms benefit	Perceived	Effort
1	13	TournamentDetail render path + memoization	mid	2-3d
2	7	mobile critical-CSS, transform-only animations	mid	1-2d
3	8	skeletons (pure perceived-perf — no real ms)	high	1-2d
4	F9.2	VM CPU bump (1 shared → 2 dedicated) — apply p95 ceiling	low	$$ ops
5	aux	Better Auth rate-limit whitelist (synthetic only)	none	0.5d

(Phase 1c promoted to Tier 0 on 2026-05-05 — the dedupe is cheap enough to ship in parallel with Phase 1, no longer waits on it.)

1. **Phase 13 — Tournament page** is the heaviest single surface (1900-line render path). Phase 1 should already lazy-load it, but the page itself still needs Suspense splitting + memoization + pagination on the round/match lists. Affects only users on that route, but they're our most-engaged cohort.
2. **Phase 7 — Mobile-specific.** Mobile FCP is ~3× desktop today — parse cost on Moto G4 CPU. If Phase 1 doesn't bring it to budget on its own, critical-CSS extraction + transform-only animations are the next levers. Pairs with Phase 1; only ranks below 13 because it's a "mobile remainder" — needs Phase 1 to land first to know the actual residual.
3. **Phase 8 — Skeletons everywhere new** (TournamentDetailPage, TournamentsPage, BotProfilePage, ProfilePage). Pure perceived-perf pass — the actual route isn't faster, just feels faster while chunks load. Pairs with Phase 1's `<Suspense>` boundaries (the skeleton *is* the fallback). Real-ms impact = 0.
4. **F9.2 — VM CPU bump (1 shared → 2 dedicated).** Direct test of the leading hypothesis for why F9.1's pool fix didn't budge apply p95 at c=25. Predicted apply p95 c=25: 46 → 15-20ms. Budget call, not a code change — track separately. Validate with `perf-sse-load --concurrency=25,50` before/after.
5. **Better Auth rate-limit whitelist for synthetic.** `get-session` 90/200 ok at concurrency 5 in prod. Real users won't hit this; the synthetic harness needs an exemption (allowlist by IP or signed header) so future perf runs aren't polluted by 429s. **Measurement hygiene only — no user-visible benefit.**

(Phase 0.2 RUM dropped from this list — D1 RUM beacon and admin Web Vitals dashboards already shipped via `f6b7079` and `507f7f6`.)

Quick wins — high leverage, can run alongside any tier

These don't need to wait for Tier 0 and don't conflict with later work. Order by gut-feel impact:

- **Service Worker app shell** (Phase 20). Repeat-visit Ready ≈ 0ms. Highest-leverage perceived-perf change in the plan.
- **Speculation Rules API** (Section E1). Native Chromium prerender for same-origin links — zero JS, zero library, instant nav on every cold-anon Chromium visitor.
- **103 Early Hints** (Section E1). Preload critical chunks before TTFB. 50–150ms shaved off cold first paint, free with Fly support.
- **Critical CSS inlined into `<head>`** (Phase 7). Cheap mobile FCP win the platform never had.
- **Materialized view for the leaderboard** (Phase 2). Kills a hot DB path forever; simple Postgres feature.
- **WASM SIMD minimax** (Section E3). Bot move latency from ~30ms to ~1ms. Becomes load-bearing once Connect4 lands.
- **CSS `aspect-ratio` + BlurHash placeholders** (Section E1). Free CLS kill + perceived-instant images.
- **`content-visibility: auto`** on long off-screen sections (Phase 7, Phase 13). One CSS line per container; free render skip.
- **Brotli quality 11** (Section E1) + **Brotli at the edge** (Phase 1). Cumulative 15–25% smaller wire bytes for zero runtime cost.
- **Atomize Zustand stores** (Section E2). One pattern fix; large re-render cut on TournamentDetail under live SSE updates.
- **`scheduler.yield()` in heavy loops** (Section E1). Pulls INP under 100ms on Moto G4 with no algorithmic change.
- **Variable fonts + `font-display: optional`** (Section E1). ~50% smaller font weight + never-blocking render.
- **Pre-warm caches on boot** (Phase 2). Built-in bots, puzzles, system config. The first user stops paying the miss.
- **Resource hints for OAuth providers** (Section E1). Sign-in feels ~200ms faster on cold visitors.

Tier 2 — promote *only* when data shows them as a bottleneck

These were Tier 1 in the original draft but the snapshot doesn't support attacking them yet. Promote to Tier 1 only when re-measurement shows them on the critical path.

- **Phase 2** — DB index pass. Backend p95 confirmed healthy across five endpoints in the 2026-05-05 baseline (no endpoint p95 over 140 ms). Until RED metrics show DB time > target on a real surface, this stays speculative.
- **Phase 4** — Cross-service network shape. Needs measurement before it earns a slot.
- **Phase 6** — Backend cold start. Now in iad with `auto_stop_machines` policy in place from `29542f7`. Needs cold-start measurement to decide whether the policy is worth the cost.
- **Phase 1b** — PlayVsBot deep-dive. *Demoted from Tier 0 on 2026-05-05.* Latest baseline shows PlayVsBot Ready at 991 ms desktop / 2058 ms mobile — within the same band as every other route. Re-promote only if post-Phase-1 numbers show PlayVsBot specifically lagging.

(Phase 5 was here in the 0.4 draft. Promoted to Tier 0 — see above.)

Tier 3 — perceived-perf polish, after the floor moves

These look like wins on a fast platform but reading "Refreshing…" on a 2-second cold page just adds noise. Land after Tier 0 + Tier 1.

- Phase 9 (optimistic / streaming everywhere)
- Phase 10 (hover prefetch on tournament cards / bot rows)
- Phase 11 (route transition polish — view-transitions API)
- Phase 12 (in-game animation budgets)
- Phase 14 (Gym worker-thread inference + WASM)
- Phase 15 (Guide INP audit)
- Phase 16 (Tables-as-primitive overhead)
- Phase 18 (production perf dashboard) — wait for RUM data first.

Tier 4 — architecture experiments (Phase 21 + Section E7)

These are re-architectures, not phases. Pursue only if Tier 0 + Tier 1 + Tier 2 + Tier 3 data still shows the gap they would close.

- **Bun runtime for backend + tournament** (E7). Collapses Phase 6 cold-start entirely; ~30% throughput gain; biggest single win in this tier.
- **Hono on the edge for read paths** (Phase 21 / E7). Promote first if Bun runs into compatibility issues.

- **WebTransport replacing SSE** (E4). Lower latency + better mobile connection migration. Big lift; only after RUM data.
- **WebRTC peer-to-peer for PvP** (E4). Move latency drops to one same-network round-trip once tables are seated. PvP-only payoff.
- **Cloudflare Durable Objects for tablePresence** (E7). Frees backend to scale horizontally.
- **SSR / RSC for `/`** (Phase 21). Cheaper alternatives in Tier 0/1 may close the gap first.
- **Edge functions for hot reads** (Phase 21). Subsumed by Edge KV in E4 if that lands first.
- **gRPC-Web on hot endpoints** (Phase 21 / E7). Revisit after Bun.
- **Self-rolled JWT replacing Better Auth** (E7). Only with measurement showing the auth library is the bottleneck.
- **Preact compat alias** (longshot from Phase 1).

Re-measure cadence

- After Tier 0 lands → run `perf-v2.js --target=staging --warmup` and append a new `Performance_Snapshot_<date>.md`.
- After each Tier 1 phase → same.
- Tier 2 phases require Phase 0.3 RED metrics to be in place first.

Let the data drive every promotion. If a phase lands and the next snapshot doesn't show movement, the phase is *landed but ineffective* and the assumption gets revisited.